



COMP 352 – Final Project: Footballer Market Evaluation

Goal: Predict a footballer's **market value next year** using **only historical information** (no future leakage).

Authors: Zevin Attisha, Christian Vaikona, Santiago Guerrero, Santiago Pedetti

Dataset source: <https://www.kaggle.com/datasets/davidcariboo/player-scores>

Table of Contents

1. Environment Setup
2. Data Importing and Pre-processing
3. Data Analysis and Visualization
4. Data Analytics (Modeling)
5. Summary / Conclusions

1. Data Importing and Pre-processing (100 Points)

- Import dataset and describe characteristics such as dimensions, data types, file types, and import methods used
- Clean, wrangle, and handle missing data, duplicate data, etc.
- Encode any categorical variables
- Perform feature engineering on the dataset
- Transform data appropriately using techniques such as aggregation, normalization, and feature construction
- Reduce redundant data and perform need based discretization

We load two datasets:

- `player_valuations.csv` : multiple valuation records per player over time
- `players.csv` : one record per player with demographic and categorical info

We then clean data, engineer historical features, and build a **time-based target**: next year's market value.

Environment Setup

This notebook uses common Python data science libraries (pandas, numpy, matplotlib, scikit-learn).

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, TimeSeriesSplit, cross_val_score
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.linear_model import LinearRegression, ElasticNet
from sklearn.ensemble import RandomForestRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score

pd.set_option("display.max_columns", None)
np.set_printoptions(suppress=True)
```

```
In [2]: PATH_PLAYER_VALUATIONS = "data/raw/player_valuations.csv"
PATH_PLAYERS = "data/raw/players.csv"
PATH_APPEARANCES = "data/raw/appearances.csv"
PATH_LINEUPS = "data/raw/game_lineups.csv"

player_valuations_df = pd.read_csv(PATH_PLAYER_VALUATIONS)
players_df = pd.read_csv(PATH_PLAYERS)
appearances_df = pd.read_csv(PATH_APPEARANCES)
lineups_df = pd.read_csv(PATH_LINEUPS)

print("player_valuations_df shape:", player_valuations_df.shape)
print("players_df shape:", players_df.shape)
print("appearances_df shape:", appearances_df.shape)
print("lineups_df shape:", lineups_df.shape)

print("player_valuations_df:")
display(player_valuations_df.head())

print("players_df:")
display(players_df.head())

print("appearances_df:")
display(appearances_df.head())

print("lineups_df:")
display(lineups_df.head())
```

player_valuations_df shape: (496606, 5)

players_df shape: (32601, 23)

appearances_df shape: (1706806, 13)

lineups_df shape: (2191911, 10)

player_valuations_df:

	player_id	date	market_value_in_eur	current_club_id	player_club_domest
0	405973	2000-01-20	150000	3057	
1	342216	2001-07-20	100000	1241	
2	3132	2003-12-09	400000	126	
3	6893	2003-12-15	900000	984	
4	10	2004-10-04	7000000	398	

players_df:

	player_id	first_name	last_name	name	last_season	current_club_id	player_club_domest
0	10	Miroslav	Klose	Miroslav Klose	2015	398	
1	26	Roman	Weidenfeller	Roman Weidenfeller	2017	16	w
2	65	Dimitar	Berbatov	Dimitar Berbatov	2015	1091	
3	77	NaN	Lúcio	Lúcio	2012	506	
4	80	Tom	Starke	Tom Starke	2017	27	

appearances_df:

	appearance_id	game_id	player_id	player_club_id	player_current_club_id	player_club_domest
0	2231978_38004	2231978	38004	853	235	201
1	2233748_79232	2233748	79232	8841	2698	201
2	2234413_42792	2234413	42792	6251	465	201
3	2234418_73333	2234418	73333	1274	6646	201
4	2234421_122011	2234421	122011	195	3008	201

lineups_df:

	game_lineups_id	date	game_id	player_id	club_id	pl
0	b2dbe01c3656b06c8e23e9de714e26bb	2013-07-27	2317258	1443	610	
1	b50a3ec6d52fd1490aab42042ac4f738	2013-07-27	2317258	5017	610	
2	7d890e6d0ff8af84b065839966a0ec81	2013-07-27	2317258	9602	1090	
3	8c355268678b9bbc7084221b1f0fde36	2013-07-27	2317258	12282	610	
4	76193074d549e5fdce4cdcbba0d66247	2013-07-27	2317258	25427	1090	R

```
In [3]: # ===== Duplicate Check =====
print("=" * 60)
print("=== Duplicate Data Check ===")
print("=" * 60)

print("\nPlayer Valuations Dataset:")
print(f"Total rows: {len(player_valuations_df)}")
print(f"Duplicate rows: {player_valuations_df.duplicated().sum()}")
if player_valuations_df.duplicated().sum() > 0:
    print(f"Removing {player_valuations_df.duplicated().sum()} duplicate rows.")
    player_valuations_df = player_valuations_df.drop_duplicates()
    print(f"Rows after removing duplicates: {len(player_valuations_df)}")
else:
    print("No duplicate rows found.")

print("\nPlayers Dataset:")
print(f"Total rows: {len(players_df)}")
print(f"Duplicate rows: {players_df.duplicated().sum()}")
if players_df.duplicated().sum() > 0:
    print(f"Removing {players_df.duplicated().sum()} duplicate rows...")
    players_df = players_df.drop_duplicates()
    print(f"Rows after removing duplicates: {len(players_df)}")
else:
    print("No duplicate rows found.")
```

```
=====
=== Duplicate Data Check ===
=====
```

```
Player Valuations Dataset:
Total rows: 496606
Duplicate rows: 0
No duplicate rows found.
```

```
Players Dataset:
Total rows: 32601
Duplicate rows: 0
No duplicate rows found.
```

```

In [4]: print("player_valuations_df dtypes:")
display(player_valuations_df.dtypes)

print("\nplayers_df dtypes:")
display(players_df.dtypes)

print("\nappearances_df dtypes:")
display(appearances_df.dtypes)

# Columns that display the most missing values in descending order

print("\nMissing values (top 10) - player_valuations_df:")
display(player_valuations_df.isna().sum().sort_values(ascending=False).head(10))

print("\nMissing values (top 10) - players_df:")
display(players_df.isna().sum().sort_values(ascending=False).head(10))

print("\nMissing values - appearances_df:")
display(appearances_df.isna().sum())
print("\nSample of appearances data:")
display(appearances_df.head(10))

```

```

player_valuations_df dtypes:
player_id          int64
date              object
market_value_in_eur  int64
current_club_id    int64
player_club_domestic_competition_id  object
dtype: object
players_df dtypes:
player_id          int64
first_name         object
last_name          object
name               object
last_season        int64
current_club_id    int64
player_code        object
country_of_birth   object
city_of_birth       object
country_of_citizenship  object
date_of_birth       object
sub_position        object
position            object
foot                object
height_in_cm        float64
contract_expiration_date  object
agent_name          object
image_url           object
url                 object
current_club_domestic_competition_id  object
current_club_name    object
market_value_in_eur  float64
highest_market_value_in_eur  float64
dtype: object

```

```

appearances_df dtypes:
appearance_id      object
game_id            int64
player_id           int64
player_club_id     int64
player_current_club_id int64
date               object
player_name        object
competition_id     object
yellow_cards       int64
red_cards          int64
goals              int64
assists            int64
minutes_played     int64
dtype: object
Missing values (top 10) - player_valuations_df:
player_id      0
date           0
market_value_in_eur 0
current_club_id 0
player_club_domestic_competition_id 0
dtype: int64
Missing values (top 10) - players_df:
agent_name      16019
contract_expiration_date 12091
country_of_birth 2799
foot            2536
city_of_birth   2455
height_in_cm    2256
first_name      2062
highest_market_value_in_eur 1523
market_value_in_eur 1523
country_of_citizenship 383
dtype: int64
Missing values - appearances_df:
appearance_id      0
game_id            0
player_id           0
player_club_id     0
player_current_club_id 0
date               0
player_name        6
competition_id     0
yellow_cards       0
red_cards          0
goals              0
assists            0
minutes_played     0
dtype: int64
Sample of appearances data:

```

	appearance_id	game_id	player_id	player_club_id	player_current_club_id	
0	2231978_38004	2231978	38004	853	235	201
1	2233748_79232	2233748	79232	8841	2698	201
2	2234413_42792	2234413	42792	6251	465	201
3	2234418_73333	2234418	73333	1274	6646	201
4	2234421_122011	2234421	122011	195	3008	201
5	2234421_146889	2234421	146889	195	2778	201
6	2235539_28716	2235539	28716	282	7185	201
7	2235539_69445	2235539	69445	282	19771	201
8	2235545_19409	2235545	19409	317	200	201
9	2235545_30003	2235545	30003	317	317	201

```
In [5]: # Clean appearances data
app = appearances_df.copy()

# Parse date to datetime
app['date'] = pd.to_datetime(app['date'], errors='coerce')
app = app.dropna(subset=['date'])

# Extract year from match date
app['year'] = app['date'].dt.year

# Fill missing performance stats with 0 (player appeared but didn't score/assist)
app['goals'] = app['goals'].fillna(0)
app['assists'] = app['assists'].fillna(0)
app['minutes_played'] = app['minutes_played'].fillna(0)
app['yellow_cards'] = app['yellow_cards'].fillna(0)
app['red_cards'] = app['red_cards'].fillna(0)

print("app shape after cleaning:", app.shape)
print("\nSample:")
display(app[['player_id', 'date', 'year', 'goals', 'assists', 'minutes_played']])
```

app shape after cleaning: (1706806, 14)

Sample:

	player_id	date	year	goals	assists	minutes_played
0	38004	2012-07-03	2012	2	0	90
1	79232	2012-07-05	2012	0	0	90
2	42792	2012-07-05	2012	0	0	45
3	73333	2012-07-05	2012	0	0	90
4	122011	2012-07-05	2012	0	1	90
5	146889	2012-07-05	2012	0	0	90
6	28716	2012-07-05	2012	0	0	90
7	69445	2012-07-05	2012	0	1	90
8	19409	2012-07-05	2012	0	0	45
9	30003	2012-07-05	2012	0	0	90

```
In [6]: print("="*80)
print("CREATING LIFETIME CUMULATIVE PERFORMANCE STATISTICS ")
print("="*80)

app = appearances_df.copy()

# Parse date to datetime
app['date'] = pd.to_datetime(app['date'], errors='coerce')
app = app.dropna(subset=['date'])

# Fill missing performance stats with 0 (player appeared but didn't score/assist)
app['goals'] = app['goals'].fillna(0)
app['assists'] = app['assists'].fillna(0)
app['minutes_played'] = app['minutes_played'].fillna(0)
app['yellow_cards'] = app['yellow_cards'].fillna(0)
app['red_cards'] = app['red_cards'].fillna(0)

# Sort by player and date
app = app.sort_values(['player_id', 'date']).reset_index(drop=True)

print("\nCalculating LIFETIME cumulative statistics...")
print(" (Each player's cumulative stats span their entire career)")

app['cumulative_goals'] = app.groupby(['player_id'])['goals'].cumsum()
app['cumulative_assists'] = app.groupby(['player_id'])['assists'].cumsum()
app['cumulative_minutes'] = app.groupby(['player_id'])['minutes_played'].cumsum()
app['cumulative_yellows'] = app.groupby(['player_id'])['yellow_cards'].cumsum()
app['cumulative_reds'] = app.groupby(['player_id'])['red_cards'].cumsum()
app['cumulative_appearances'] = app.groupby(['player_id']).cumcount() + 1

# Keep only the most recent row per player per date (in case multiple games same date)
app_cumulative = app.groupby(['player_id', 'date']).tail(1).reset_index(drop=True)

# Select columns for merge
```



```

app_for_merge = app_cumulative[[
    'player_id', 'date',
    'cumulative_goals', 'cumulative_assists', 'cumulative_minutes',
    'cumulative_yellows', 'cumulative_reds', 'cumulative_appearances'
]].copy()

print(f"\n app_for_merge shape: {app_for_merge.shape}")
print(f" Date range: {app_for_merge['date'].min()} to {app_for_merge['date'].max()}")
print(f" Unique players: {app_for_merge['player_id'].nunique()}")

# Verify cumulative logic is working correctly
print("\n Verification - checking cumulative growth for a sample player:")
sample_player_id = app_for_merge['player_id'].iloc[100] if len(app_for_merge) > 100 else app_for_merge['player_id'].iloc[0]
sample_data = app_for_merge[app_for_merge['player_id'] == sample_player_id].head(10)
print(f"\n Player ID: {sample_player_id}")
print("Notice how cumulative_goals increases over time (lifetime cumulative):")
display(sample_data[['date', 'cumulative_goals', 'cumulative_assists', 'cumulative_minutes', 'cumulative_yellows', 'cumulative_reds', 'cumulative_appearances']])

print("\n" + "="*80)
print("PERFORMANCE DATA PREPARED")
print("="*80)

```

```

=====
=
CREATING LIFETIME CUMULATIVE PERFORMANCE STATISTICS
=====
=

```

Calculating LIFETIME cumulative statistics...
 (Each player's cumulative stats span their entire career)

```

app_for_merge shape: (1706792, 8)
Date range: 2012-07-03 00:00:00 to 2025-04-10 00:00:00
Unique players: 25689

```

Verification - checking cumulative growth for a sample player:

```

Player ID: 10
Notice how cumulative_goals increases over time (lifetime cumulative):

```

	date	cumulative_goals	cumulative_assists	cumulative_minutes	cumulat
0	2012-08-23	1	0	83	
1	2012-08-26	1	0	173	
2	2012-09-02	3	0	262	
3	2012-09-16	4	0	352	
4	2012-09-20	4	0	442	
5	2012-09-23	4	0	456	
6	2012-09-26	4	0	546	
7	2012-09-30	4	1	636	
8	2012-10-07	6	1	708	
9	2012-10-20	7	1	798	

```
=====
=
PERFORMANCE DATA PREPARED
=====
=
```

We keep only the columns needed for modeling and remove columns that are not useful or could cause leakage (e.g., current market value fields in `players.csv`).

```
In [7]: pv = player_valuations_df.copy()

# Parse date
pv["date"] = pd.to_datetime(pv["date"], errors="coerce")
pv = pv.dropna(subset=["date"])

# Remove duplicates (same player_id & date)
pv = pv.drop_duplicates(subset=["player_id", "date"], keep="last")

# Keep essential columns
pv = pv[[
    "player_id",
    "date",
    "market_value_in_eur",
    "current_club_id",
    "player_club_domestic_competition_id"
]].copy()

# Rename for clarity
pv = pv.rename(columns={
    "market_value_in_eur": "market_value_in_eur_valuation",
    "current_club_id": "current_club_id_valuation",
    "player_club_domestic_competition_id": "domestic_competition_id_valuation"
})
```

```
print("pv shape:", pv.shape)
pv.head()
```

pv shape: (496606, 5)

```
Out[7]:
```

	player_id	date	market_value_in_eur_valuation	current_club_id_valuation
0	405973	2000-01-20	150000	305
1	342216	2001-07-20	100000	124
2	3132	2003-12-09	400000	120
3	6893	2003-12-15	900000	98
4	10	2004-10-04	7000000	39

```
In [8]: # Clean players (p)

p = players_df.copy()

# Parse birth date
p["date_of_birth"] = pd.to_datetime(p["date_of_birth"], errors="coerce")

# Keep only safe player-level features (no leakage)
keep_player_cols = [
    "player_id",
    "date_of_birth",
    "position",
    "sub_position",
    "foot",
    "height_in_cm"
]
p = p[keep_player_cols].copy()

# Handle missing categoricals
p["position"] = p["position"].fillna("Missing")
p["sub_position"] = p["sub_position"].fillna("Missing")
p["foot"] = p["foot"].fillna("Missing")

# Numeric height + impute
p["height_in_cm"] = pd.to_numeric(p["height_in_cm"], errors="coerce")
p["height_in_cm"] = p["height_in_cm"].fillna(p["height_in_cm"].median())

# Birth date required for age feature
p = p.dropna(subset=["date_of_birth"])

print("p shape:", p.shape)
p.head()
```

p shape: (32554, 6)

Out[8]:

	player_id	date_of_birth	position	sub_position	foot	height_in_cm
0	10	1978-06-09	Attack	Centre-Forward	right	184.0
1	26	1980-08-06	Goalkeeper	Goalkeeper	left	190.0
2	65	1981-01-30	Attack	Centre-Forward	Missing	183.0
3	77	1978-05-08	Defender	Centre-Back	Missing	183.0
4	80	1981-03-18	Goalkeeper	Goalkeeper	right	194.0

We merge valuation records with player info using `player_id`. Each valuation row becomes a modeling row with player features attached.

```
In [9]: # ===== Merge player valuations with player info =====
merged_df = pv.merge(p, on="player_id", how="left")

# Drop valuation rows missing player info
merged_df = merged_df.dropna(subset=["date_of_birth"])

print("merged_df shape:", merged_df.shape)
merged_df.head()

# ===== Yearly snapshots (1 row per player per year) =====

# Ensure datetime for valuation date
merged_df["date"] = pd.to_datetime(merged_df["date"], errors="coerce")
merged_df = merged_df.dropna(subset=["date"])

# Create a year column
merged_df["year"] = merged_df["date"].dt.year

# Keep only the last valuation within each year per player
merged_df = (
    merged_df.sort_values(["player_id", "date"])
    .groupby(["player_id", "year"], as_index=False)
    .tail(1)
    .reset_index(drop=True)
)

print("merged_df shape after yearly snapshot:", merged_df.shape)
merged_df[["player_id", "date", "year", "market_value_in_eur_valuation"]].head()

# ===== Manual merge_asof by player =====
print("\n" + "="*80)
print("MERGING PERFORMANCE STATISTICS WITH TEMPORAL INTEGRITY")
print("Using manual player-by-player merge (more reliable)")
print("="*80)

# ===== NaN player_id =====
```

```

print("="*60)
print("FIXING NaN player_id VALUES")
print("="*60)

# Diagnostic
print(f"\nBefore cleaning:")
print(f"  Total rows: {len(merged_df)}")
print(f"  NaN player_ids: {merged_df['player_id'].isna().sum()}")

# Remove rows with NaN player_id
merged_df = merged_df.dropna(subset=['player_id'])

print(f"\nAfter cleaning:")
print(f"  Total rows: {len(merged_df)}")
print(f"  NaN player_ids: {merged_df['player_id'].isna().sum()}")
print("="*60 + "\n")
# =====

# Ensure data types
merged_df['player_id'] = merged_df['player_id'].astype(int)
app_for_merge['player_id'] = app_for_merge['player_id'].astype(int)
merged_df['date'] = pd.to_datetime(merged_df['date'])
app_for_merge['date'] = pd.to_datetime(app_for_merge['date'])

# Sort data
merged_df = merged_df.sort_values(['player_id', 'date']).reset_index(drop=True)
app_for_merge = app_for_merge.sort_values(['player_id', 'date']).reset_index(drop=True)

print(f"Starting merge for {merged_df['player_id'].nunique()} unique players..")
print(f"This may take 1-2 minutes...\n")

# Manual merge by player
result_list = []
unique_players = merged_df['player_id'].unique()

import time
start_time = time.time()

for i, pid in enumerate(unique_players):
    # Progress indicator every 1000 players
    if (i + 1) % 1000 == 0:
        elapsed = time.time() - start_time
        print(f"Processed {i+1}/{len(unique_players)} players ({elapsed:.1f}s")

    # Get data for this player
    player_val = merged_df[merged_df['player_id'] == pid].copy()
    player_app = app_for_merge[app_for_merge['player_id'] == pid].copy()

    # Merge with appearances data if available
    if len(player_app) > 0:
        player_merged = pd.merge_asof(
            player_val.sort_values('date'),
            player_app.sort_values('date'),

```

```

        on='date',
        direction='backward',
        tolerance=pd.Timedelta(days=90),
        allow_exact_matches=True
    )
else:
    # No appearance data for this player - keep valuation data with NaN per
    player_merged = player_val

    result_list.append(player_merged)

# Combine all players
merged_df = pd.concat(result_list, ignore_index=True)

elapsed_total = time.time() - start_time
print(f"\n Merge complete in {elapsed_total:.1f} seconds!")
print(f"Final merged_df shape: {merged_df.shape}")
print(f"Rows with performance data: {merged_df['cumulative_goals'].notna().sum}")
print(f"Rows without performance data: {merged_df['cumulative_goals'].isna().sum}")

# Rename cumulative columns
merged_df = merged_df.rename(columns={
    'cumulative_goals': 'total_goals',
    'cumulative_assists': 'total_assists',
    'cumulative_minutes': 'total_minutes',
    'cumulative_yellows': 'total_yellow_cards',
    'cumulative_reds': 'total_red_cards',
    'cumulative_appearances': 'num_appearances'
})

# Fill NaNs (players with no appearances before valuation date)
perf_cols = ['total_goals', 'total_assists', 'total_minutes', 'num_appearances',
             'total_yellow_cards', 'total_red_cards']
for col in perf_cols:
    if col in merged_df.columns:
        merged_df[col] = merged_df[col].fillna(0)

# Calculate per-90 stats
print("\nCalculating per-90 minute statistics...")
merged_df['goals_per_90'] = np.where(
    merged_df['total_minutes'] > 0,
    (merged_df['total_goals'] / merged_df['total_minutes']) * 90,
    0
)
merged_df['assists_per_90'] = np.where(
    merged_df['total_minutes'] > 0,
    (merged_df['total_assists'] / merged_df['total_minutes']) * 90,
    0
)
merged_df['goal_contributions_per_90'] = merged_df['goals_per_90'] + merged_df['assists_per_90']
merged_df['games_started_estimate'] = merged_df['total_minutes'] / 90
merged_df['cards_per_90'] = np.where(
    merged_df['total_minutes'] > 0,

```

```

        ((merged_df['total_yellow_cards'] + merged_df['total_red_cards'] * 2) / merged_df['total_minutes'] * 60)
    )

print(f"\nPerformance statistics merged with NO TEMPORAL LEAKAGE")
print(f"Final shape: {merged_df.shape}")
print("\nSample with performance features:")
display(merged_df[['player_id', 'date', 'year', 'market_value_in_eur_valuation', 'total_goals', 'total_assists', 'total_minutes', 'goals_per_minute']])

print(f"\nPerformance statistics merged with NO TEMPORAL LEAKAGE")
print(f"Final shape: {merged_df.shape}")
print("\nSample with performance features:")
display(merged_df[['player_id', 'date', 'year', 'market_value_in_eur_valuation', 'total_goals', 'total_assists', 'total_minutes', 'goals_per_minute']])

# Safe verification - check if ANY players have performance data
players_with_perf = merged_df[merged_df['total_goals'] > 0]
if len(players_with_perf) > 0:
    print(f"\n Verification - found {len(players_with_perf)} observations with performance data")
    sample_player_id = players_with_perf['player_id'].iloc[0]
    sample_player = merged_df[merged_df['player_id'] == sample_player_id].head(1)
    display(sample_player[['player_id', 'date', 'year', 'total_goals', 'total_assists', 'total_minutes', 'goals_per_minute']])
else:
    print(f"\n No players have goals > 0 in current dataset")
    print(f"This may indicate:")
    print(f"  1. Valuations are from before appearance data starts (2012)")
    print(f"  2. Tolerance window (365 days) may need adjustment")
    print(f"  3. Player IDs may not match between datasets")

# Show date ranges
print(f"\nDate ranges:")
print(f"  Valuations: {merged_df['date'].min()} to {merged_df['date'].max()}")
print(f"  Appearances: {app_for_merge['date'].min()} to {app_for_merge['date'].max()}")

```

```
merged_df shape: (496045, 10)
merged_df shape after yearly snapshot: (251792, 11)
```

```
=====
=
MERGING PERFORMANCE STATISTICS WITH TEMPORAL INTEGRITY
Using manual player-by-player merge (more reliable)
=====
=
=====
FIXING NaN player_id VALUES
=====
```

```
Before cleaning:
  Total rows: 251792
  NaN player_ids: 0
```

```
After cleaning:
  Total rows: 251792
  NaN player_ids: 0
=====
```

```
Starting merge for 31046 unique players...
This may take 1-2 minutes...
```

```
Processed 1000/31046 players (1.4s elapsed)...
Processed 2000/31046 players (2.9s elapsed)...
Processed 3000/31046 players (4.3s elapsed)...
Processed 4000/31046 players (5.7s elapsed)...
Processed 5000/31046 players (7.2s elapsed)...
Processed 6000/31046 players (8.5s elapsed)...
Processed 7000/31046 players (10.0s elapsed)...
Processed 8000/31046 players (11.3s elapsed)...
Processed 9000/31046 players (12.8s elapsed)...
Processed 10000/31046 players (14.1s elapsed)...
Processed 11000/31046 players (15.4s elapsed)...
Processed 12000/31046 players (16.9s elapsed)...
Processed 13000/31046 players (18.2s elapsed)...
Processed 14000/31046 players (19.4s elapsed)...
Processed 15000/31046 players (20.7s elapsed)...
Processed 16000/31046 players (22.1s elapsed)...
Processed 17000/31046 players (23.4s elapsed)...
Processed 18000/31046 players (24.7s elapsed)...
Processed 19000/31046 players (25.9s elapsed)...
Processed 20000/31046 players (27.3s elapsed)...
Processed 21000/31046 players (28.6s elapsed)...
Processed 22000/31046 players (29.9s elapsed)...
Processed 23000/31046 players (31.1s elapsed)...
Processed 24000/31046 players (32.4s elapsed)...
Processed 25000/31046 players (33.7s elapsed)...
Processed 26000/31046 players (35.1s elapsed)...
Processed 27000/31046 players (36.3s elapsed)...
Processed 28000/31046 players (37.5s elapsed)...
Processed 29000/31046 players (38.7s elapsed)...
```


Processed 30000/31046 players (39.8s elapsed)...

Processed 31000/31046 players (41.1s elapsed)...

Merge complete in 45.1 seconds!

Final merged_df shape: (251792, 19)

Rows with performance data: 69253

Rows without performance data: 182539

Calculating per-90 minute statistics...

Performance statistics merged with NO TEMPORAL LEAKAGE

Final shape: (251792, 24)

Sample with performance features:

	player_id	date	year	market_value_in_eur_valuation	total_goals	total_a
0	NaN	2004-10-04	2004	7000000	0.0	
1	NaN	2005-09-30	2005	15000000	0.0	
2	NaN	2006-07-15	2006	30000000	0.0	
3	NaN	2007-06-21	2007	23000000	0.0	
4	NaN	2008-06-04	2008	20000000	0.0	
5	NaN	2009-08-30	2009	12000000	0.0	
6	NaN	2010-05-08	2010	7500000	0.0	
7	NaN	2011-06-29	2011	5000000	0.0	
8	NaN	2012-07-03	2012	6000000	0.0	
9	NaN	2013-06-19	2013	2000000	16.0	

Performance statistics merged with NO TEMPORAL LEAKAGE

Final shape: (251792, 24)

Sample with performance features:

	player_id	date	year	market_value_in_eur_valuation	total_goals	total_a
0	NaN	2004-10-04	2004	7000000	0.0	
1	NaN	2005-09-30	2005	15000000	0.0	
2	NaN	2006-07-15	2006	30000000	0.0	
3	NaN	2007-06-21	2007	23000000	0.0	
4	NaN	2008-06-04	2008	20000000	0.0	
5	NaN	2009-08-30	2009	12000000	0.0	
6	NaN	2010-05-08	2010	7500000	0.0	
7	NaN	2011-06-29	2011	5000000	0.0	
8	NaN	2012-07-03	2012	6000000	0.0	
9	NaN	2013-06-19	2013	2000000	16.0	

Verification - found 48012 observations with goals > 0:

	player_id	date	year	total_goals	total_assists	total_minutes
--	-----------	------	------	-------------	---------------	---------------

```
In [10]: print("="*80)
print("CHECKING DATA BEFORE/AFTER LINEUPS MERGE")
print("="*80)

# This should show your current state
print(f"Current merged_df shape: {merged_df.shape}")
print(f"Players with goals > 0: {(merged_df['total_goals'] > 0).sum()}")
print(f"Players with minutes > 0: {(merged_df['total_minutes'] > 0).sum()}")

# Check if you accidentally filtered your data
print(f"\nDate range in current data:")
print(f"  Min: {merged_df['date'].min()}")
print(f"  Max: {merged_df['date'].max()}")
print(f"  Unique years: {sorted(merged_df['year'].unique())}")
```

```
=====
=
CHECKING DATA BEFORE/AFTER LINEUPS MERGE
=====
```

```
=
Current merged_df shape: (251792, 24)
Players with goals > 0: 48012
Players with minutes > 0: 69253
```

Date range in current data:

Min: 2000-01-20 00:00:00

Max: 2025-04-06 00:00:00

Unique years: [np.int32(2000), np.int32(2001), np.int32(2003), np.int32(2004), np.int32(2005), np.int32(2006), np.int32(2007), np.int32(2008), np.int32(2009), np.int32(2010), np.int32(2011), np.int32(2012), np.int32(2013), np.int32(2014), np.int32(2015), np.int32(2016), np.int32(2017), np.int32(2018), np.int32(2019), np.int32(2020), np.int32(2021), np.int32(2022), np.int32(2023), np.int32(2024), np.int32(2025)]

We compute age at valuation date and create historical features (lags, trends, rolling mean). We also build the target: next year's market value.

```
In [11]: print("\n" + "="*80)
print("PROCESSING LINEUP DATA WITH TEMPORAL INTEGRITY")
print("="*80)

# Ensure datetime
lineups_df["date"] = pd.to_datetime(lineups_df["date"], errors="coerce")
lineups_df = lineups_df.dropna(subset=["date"])

# Identify starts from 'type'
lineups_df["is_start"] = (
    lineups_df["type"]
    .astype(str)
    .str.lower()
    .str.contains("start", na=False)
)

# Ensure captain flag is numeric
lineups_df["team_captain"] = lineups_df["team_captain"].fillna(0).astype(int)

# Sort by player and date
lineups_df = lineups_df.sort_values(['player_id', 'date']).reset_index(drop=True)

print("\nCalculating LIFETIME cumulative lineup statistics...")
print(" (Each player's lineup stats span their entire career)")

# Create cumulative counts by player-year
lineups_df['cumulative_matches'] = lineups_df.groupby(['player_id']).cumcount()
lineups_df['cumulative_starts'] = lineups_df.groupby(['player_id'])['is_start'].cumsum()
lineups_df['cumulative_captain'] = lineups_df.groupby(['player_id'])['team_captain'].cumsum()

# Keep most recent row per player-date (in case of multiple games same day)
```

```

lineups_cumulative = lineups_df.groupby(['player_id', 'date']).tail(1).reset_i

# Select columns for merge
lineups_for_merge = lineups_cumulative[[
    'player_id', 'date',
    'cumulative_matches', 'cumulative_starts', 'cumulative_captain'
]].copy()

print(f"Prepared lineup data: {lineups_for_merge.shape}")
print("Starting player-by-player merge...")

# Drop old lineup columns if they exist (from previous run)
lineup_cols_to_drop = ['lineup_matches', 'starts', 'captain_matches', 'start_r
for col in lineup_cols_to_drop:
    if col in merged_df.columns:
        merged_df = merged_df.drop(columns=[col])
        print(f"Dropped existing column: {col}")

# fix FOR NaN player_id

print("\n" + "="*80)
print("COMPREHENSIVE PLAYER_ID DIAGNOSTIC AND FIX")
print("="*80)

# Check what columns we have
print("\n1. Current columns in merged_df:")
player_cols = [c for c in merged_df.columns if 'player' in c.lower() and 'id'
print(f"    Player ID columns found: {player_cols}")

# SCheck each column for issues
for col in player_cols:
    print(f"\n2. Analyzing {col}:")
    print(f"    - Type: {merged_df[col].dtype}")
    print(f"    - Total values: {len(merged_df[col])}")
    print(f"    - NaN count: {merged_df[col].isna().sum()}")
    print(f"    - Inf count: {np.isinf(merged_df[col].fillna(0)).sum()}")
    print(f"    - Unique values: {merged_df[col].nunique()}")
    if merged_df[col].notna().any():
        print(f"    - Sample non-NaN values: {merged_df[col].dropna().head(3).t

# Fix duplicate player_id columns if they exist
if len(player_cols) > 1:
    print(f"\n3. Multiple player_id columns detected - consolidating...")

    # Use the first non-NaN value across all player_id columns
    merged_df['player_id_fixed'] = merged_df[player_cols[0]]

    for col in player_cols[1:]:
        merged_df['player_id_fixed'] = merged_df['player_id_fixed'].fillna(mer

# Drop all old player_id columns
merged_df = merged_df.drop(columns=player_cols)

```

```

# Rename to player_id
merged_df = merged_df.rename(columns={'player_id_fixed': 'player_id'})

print(f"    Consolidated {len(player_cols)} columns into one")
print(f"    - NaN count after consolidation: {merged_df['player_id'].isna()}

#More aggressive cleaning
print("\n4. Cleaning player_id column:")
print(f"    Before: {len(merged_df)} rows")

# Remove NaN
merged_df = merged_df[merged_df['player_id'].notna()].copy()
print(f"    After removing NaN: {len(merged_df)} rows")

# Remove Inf
if merged_df['player_id'].dtype == 'float64':
    merged_df = merged_df[~np.isinf(merged_df['player_id'])].copy()
    print(f"    After removing Inf: {len(merged_df)} rows")

# Remove negative values (player IDs should be positive)
merged_df = merged_df[merged_df['player_id'] > 0].copy()
print(f"    After removing non-positive: {len(merged_df)} rows")

# Final verification
print("\n5. Final verification before conversion:")
print(f"    - Total rows: {len(merged_df)}")
print(f"    - NaN count: {merged_df['player_id'].isna().sum()}")
print(f"    - Inf count: {np.isinf(merged_df['player_id']).sum()}")
print(f"    - Min value: {merged_df['player_id'].min()}")
print(f"    - Max value: {merged_df['player_id'].max()}")
print(f"    - Data type: {merged_df['player_id'].dtype}")

# Safe conversion to int
try:
    # First convert to nullable Int64 which handles edge cases better
    merged_df['player_id'] = pd.to_numeric(merged_df['player_id'], errors='coerce')
    merged_df = merged_df[merged_df['player_id'].notna()].copy()
    merged_df['player_id'] = merged_df['player_id'].astype('Int64')
    print("\n    Successfully converted to Int64")
except Exception as e:
    print(f"\n    Error: {e}")
    print("    Attempting alternative conversion...")

    # Alternative: round first, then convert
    merged_df['player_id'] = merged_df['player_id'].round(0)
    merged_df['player_id'] = merged_df['player_id'].astype(int)
    print("    Successfully converted using alternative method")

print(f"\n6. After conversion:")
print(f"    - Data type: {merged_df['player_id'].dtype}")
print(f"    - Sample values: {merged_df['player_id'].head(5).tolist()}")

# Also fix lineups_for_merge

```

```

print("\n7. Fixing lineups_for_merge player_id:")
lineups_for_merge = lineups_for_merge[lineups_for_merge['player_id'].notna()].
lineups_for_merge['player_id'] = pd.to_numeric(lineups_for_merge['player_id'],
lineups_for_merge = lineups_for_merge[lineups_for_merge['player_id'].notna()].
lineups_for_merge['player_id'] = lineups_for_merge['player_id'].astype('Int64')
print(f"    lineups_for_merge player_id fixed")

print("="*80)
print("PLAYER_ID CONVERSION COMPLETE - READY FOR MERGE")
print("="*80 + "\n")

# Manual merge by player
import time
start_time = time.time()

result_list = []
unique_players = merged_df['player_id'].unique()

print(f"Starting merge for {len(unique_players)} unique players...")

for i, pid in enumerate(unique_players):
    if (i + 1) % 1000 == 0:
        elapsed = time.time() - start_time
        print(f"Processed {i+1}/{len(unique_players)} players ({elapsed:.1f}s

    player_val = merged_df[merged_df['player_id'] == pid].copy()
    player_lineup = lineups_for_merge[lineups_for_merge['player_id'] == pid].c

    if len(player_lineup) > 0:
        # Drop player_id before merge_asof to avoid duplication
        player_lineup_for_asof = player_lineup[['date', 'cumulative_matches',
                                                'cumulative_starts', 'cumulat

        player_merged = pd.merge_asof(
            player_val.sort_values('date'),
            player_lineup_for_asof.sort_values('date'),
            on='date',
            direction='backward',
            tolerance=pd.Timedelta(days=90),
            allow_exact_matches=True
        )
    else:
        player_merged = player_val

    result_list.append(player_merged)

merged_df = pd.concat(result_list, ignore_index=True)

elapsed_total = time.time() - start_time
print(f"\nLineup merge complete in {elapsed_total:.1f} seconds!")
print(f"Final shape: {merged_df.shape}")
print(f"Unique players: {merged_df['player_id'].nunique()}")

```

```

# Verification
print("\n" + "="*80)
print("VERIFICATION")
print("="*80)

player_id_cols_after = [c for c in merged_df.columns if 'player' in c.lower()]
print(f"\nPlayer ID columns after merge: {player_id_cols_after}")

if len(player_id_cols_after) == 1 and player_id_cols_after[0] == 'player_id':
    print("Single clean player_id column")
else:
    print(f"WARNING: Found {len(player_id_cols_after)} player_id columns: {player_id_cols_after}")

# Check for NaN player_ids
print(f"\nNaN player_ids: {merged_df['player_id'].isna().sum()}")

# Rename and fill lineup columns
print("\n" + "="*80)
print("FINALIZING LINEUP FEATURES")
print("="*80)

# Rename cumulative columns
rename_map = {
    'cumulative_matches': 'lineup_matches',
    'cumulative_starts': 'starts',
    'cumulative_captain': 'captain_matches'
}
cols_to_rename = {k: v for k, v in rename_map.items() if k in merged_df.columns}
if cols_to_rename:
    merged_df = merged_df.rename(columns=cols_to_rename)
    print(f"Renamed columns: {list(cols_to_rename.values())}")

# Fill NaNs
for c in ["lineup_matches", "starts", "captain_matches"]:
    if c in merged_df.columns:
        merged_df[c] = merged_df[c].fillna(0)

# Calculate start ratio
if 'lineup_matches' in merged_df.columns and 'starts' in merged_df.columns:
    merged_df['start_ratio'] = np.where(
        merged_df['lineup_matches'] > 0,
        merged_df['starts'] / merged_df['lineup_matches'],
        0
    )
    merged_df['start_ratio'] = merged_df['start_ratio'].fillna(0)

print(f"\nLineup features added successfully")
print(f"Final merged_df shape: {merged_df.shape}")
print(f"Rows with lineup data: {(merged_df.get('lineup_matches', pd.Series([0]

```

```
=====
=
PROCESSING LINEUP DATA WITH TEMPORAL INTEGRITY
=====
=
```

```
Calculating LIFETIME cumulative lineup statistics...
  (Each player's lineup stats span their entire career)
Prepared lineup data: (2191895, 5)
Starting player-by-player merge...
```

```
=====
=
COMPREHENSIVE PLAYER_ID DIAGNOSTIC AND FIX
=====
=
```

1. Current columns in merged_df:
Player ID columns found: ['player_id_x', 'player_id_y', 'player_id']
2. Analyzing player_id_x:
 - Type: float64
 - Total values: 251792
 - NaN count: 26898
 - Inf count: 0
 - Unique values: 25304
 - Sample non-NaN values: [10.0, 10.0, 10.0]
2. Analyzing player_id_y:
 - Type: float64
 - Total values: 251792
 - NaN count: 182539
 - Inf count: 0
 - Unique values: 20753
 - Sample non-NaN values: [10.0, 10.0, 10.0]
2. Analyzing player_id:
 - Type: float64
 - Total values: 251792
 - NaN count: 224894
 - Inf count: 0
 - Unique values: 5742
 - Sample non-NaN values: [162.0, 162.0, 162.0]
3. Multiple player_id columns detected - consolidating...
Consolidated 3 columns into one
 - NaN count after consolidation: 0
4. Cleaning player_id column:
Before: 251792 rows
After removing NaN: 251792 rows
After removing Inf: 251792 rows
After removing non-positive: 251792 rows

5. Final verification before conversion:

- Total rows: 251792
- NaN count: 0
- Inf count: 0
- Min value: 10.0
- Max value: 1310513.0
- Data type: float64

Successfully converted to Int64

6. After conversion:

- Data type: Int64
- Sample values: [10, 10, 10, 10, 10]

7. Fixing lineups_for_merge player_id:

lineups_for_merge player_id fixed

```
=====
=
PLAYER_ID CONVERSION COMPLETE - READY FOR MERGE
=====
=
```

Starting merge for 31046 unique players...

```
Processed 1000/31046 players (2.0s elapsed)...
Processed 2000/31046 players (3.9s elapsed)...
Processed 3000/31046 players (5.9s elapsed)...
Processed 4000/31046 players (7.7s elapsed)...
Processed 5000/31046 players (9.6s elapsed)...
Processed 6000/31046 players (11.5s elapsed)...
Processed 7000/31046 players (13.4s elapsed)...
Processed 8000/31046 players (15.4s elapsed)...
Processed 9000/31046 players (17.4s elapsed)...
Processed 10000/31046 players (19.4s elapsed)...
Processed 11000/31046 players (21.4s elapsed)...
Processed 12000/31046 players (23.2s elapsed)...
Processed 13000/31046 players (25.2s elapsed)...
Processed 14000/31046 players (27.1s elapsed)...
Processed 15000/31046 players (29.0s elapsed)...
Processed 16000/31046 players (31.0s elapsed)...
Processed 17000/31046 players (33.1s elapsed)...
Processed 18000/31046 players (35.0s elapsed)...
Processed 19000/31046 players (36.9s elapsed)...
Processed 20000/31046 players (38.8s elapsed)...
Processed 21000/31046 players (40.9s elapsed)...
Processed 22000/31046 players (42.8s elapsed)...
Processed 23000/31046 players (44.7s elapsed)...
Processed 24000/31046 players (46.6s elapsed)...
Processed 25000/31046 players (48.5s elapsed)...
Processed 26000/31046 players (50.7s elapsed)...
Processed 27000/31046 players (52.7s elapsed)...
Processed 28000/31046 players (54.7s elapsed)...
Processed 29000/31046 players (56.6s elapsed)...
Processed 30000/31046 players (58.4s elapsed)...
Processed 31000/31046 players (60.1s elapsed)...
```

Lineup merge complete in 63.0 seconds!
Final shape: (251792, 25)
Unique players: 31046

```
=====
=
VERIFICATION
=====
=
```

Player ID columns after merge: ['player_id']
Single clean player_id column

NaN player_ids: 0

```
=====
=
FINALIZING LINEUP FEATURES
=====
=
```

Renamed columns: ['lineup_matches', 'starts', 'captain_matches']

Lineup features added successfully
Final merged_df shape: (251792, 26)
Rows with lineup data: 71135

```
In [12]: # Feature Engineering (time series per player, yearly)
# At this point, merged_df has ONE row per player per year (yearly snapshot).
# We now compute age, create a next-year target, and build historical lag feat

merged_df = merged_df.sort_values(["player_id", "year"]).reset_index(drop=True)

# Age at time of valuation
merged_df["age_at_valuation"] = ((merged_df["date"] - merged_df["date_of_birth"])

# Target: next year's value (t+1 year)
merged_df["market_value_next_year"] = (
    merged_df.groupby("player_id")["market_value_in_eur_valuation"].shift(-1)
)

# Historical lags (past only)
merged_df["market_value_lag_1"] = (
    merged_df.groupby("player_id")["market_value_in_eur_valuation"].shift(1)
)
merged_df["market_value_lag_2"] = (
    merged_df.groupby("player_id")["market_value_in_eur_valuation"].shift(2)
)

# Dynamics
merged_df["market_value_trend"] = merged_df["market_value_lag_1"] - merged_df["

# Rolling mean (past only) - use shift(1) then rolling window
merged_df["market_value_mean_3"] = (
```

```

merged_df.groupby("player_id")["market_value_in_eur_valuation"]
    .shift(1)
    .rolling(window=3)
    .mean()
)

# Create performance lag features (historical performance)
merged_df['goals_lag_1'] = merged_df.groupby('player_id')['total_goals'].shift(1)
merged_df['assists_lag_1'] = merged_df.groupby('player_id')['total_assists'].shift(1)
merged_df['minutes_lag_1'] = merged_df.groupby('player_id')['total_minutes'].shift(1)
merged_df['goals_per_90_lag_1'] = merged_df.groupby('player_id')['goals_per_90'].shift(1)

# Performance trends (year-over-year changes)
merged_df['goals_trend'] = merged_df['total_goals'] - merged_df['goals_lag_1']
merged_df['minutes_trend'] = merged_df['total_minutes'] - merged_df['minutes_lag_1']

# Fill NaNs in performance features
perf_lag_cols = ['goals_lag_1', 'assists_lag_1', 'minutes_lag_1',
                 'goals_per_90_lag_1', 'goals_trend', 'minutes_trend']
for col in perf_lag_cols:
    merged_df[col] = merged_df[col].fillna(0)

print("Performance lag features created")

# Drop rows without enough history or target
# Drop rows without ESSENTIAL data only (more lenient)
print(f"\nBefore dropping rows: {merged_df.shape}")

# Only require target and at least 1 lag (not 2 or 3)
merged_df = merged_df.dropna(subset=[
    "market_value_next_year",
    "market_value_lag_1", # At least 1 year of history
    "age_at_valuation"
])

print(f"After dropping rows without target/lag1/age: {merged_df.shape}")

# Fill remaining NaN features with 0 (lag_2, mean_3, etc.)
lag_fill_cols = ["market_value_lag_2", "market_value_mean_3", "market_value_trend"]
for col in lag_fill_cols:
    if col in merged_df.columns:
        merged_df[col] = merged_df[col].fillna(0)

print(f"Filled NaN lag features with 0")
print(f"Final shape after FE: {merged_df.shape}")

print("merged_df shape after FE:", merged_df.shape)
merged_df[["player_id", "year", "market_value_in_eur_valuation", "market_value_next_year", "goals", "assists", "minutes", "goals_per_90", "goals_lag_1", "assists_lag_1", "minutes_lag_1", "goals_per_90_lag_1", "goals_trend", "minutes_trend"]]

```

Performance lag features created

Before dropping rows: (251792, 38)

After dropping rows without target/lag1/age: (191153, 38)

Filled NaN lag features with 0

Final shape after FE: (191153, 38)

merged_df shape after FE: (191153, 38)

```
Out[12]:
```

	player_id	year	market_value_in_eur_valuation	market_value_next_year	ma
1	10	2005	15000000	30000000.0	
2	10	2006	30000000	23000000.0	
3	10	2007	23000000	20000000.0	
4	10	2008	20000000	12000000.0	
5	10	2009	12000000	7500000.0	
6	10	2010	7500000	5000000.0	
7	10	2011	5000000	6000000.0	
8	10	2012	6000000	2000000.0	
9	10	2013	2000000	1000000.0	
10	10	2014	1000000	1000000.0	

```
In [13]: # Feature Discretization
print("=" * 60)
print("=== Feature Discretization ===")
print("=" * 60)

# Age groups based on football career stages
merged_df['age_group'] = pd.cut(
    merged_df['age_at_valuation'],
    bins=[0, 23, 28, 33, 50],
    labels=['Young (≤23)', 'Prime (24-28)', 'Experienced (29-33)', 'Veteran (34-50)']
)

# Valuation tiers
merged_df['value_tier'] = pd.cut(
    merged_df['market_value_in_eur_valuation'],
    bins=[0, 1e6, 5e6, 20e6, float('inf')],
    labels=['Low (<1M)', 'Medium (1-5M)', 'High (5-20M)', 'Elite (>20M)']
)

# Target value tiers (for next year prediction)
merged_df['target_value_tier'] = pd.cut(
    merged_df['market_value_next_year'],
    bins=[0, 1e6, 5e6, 20e6, float('inf')],
    labels=['Low (<1M)', 'Medium (1-5M)', 'High (5-20M)', 'Elite (>20M)']
)

print("\nSample of discretized features:")
```

```
print(merged_df[['player_id', 'age_at_valuation', 'age_group',  
                'market_value_in_eur_valuation', 'value_tier']].head(10))  
  
print(f"\nAge Group Distribution:")  
print(merged_df['age_group'].value_counts().sort_index())  
  
print(f"\nCurrent Value Tier Distribution:")  
print(merged_df['value_tier'].value_counts().sort_index())  
  
print(f"\nTarget Value Tier Distribution:")  
print(merged_df['target_value_tier'].value_counts().sort_index())
```

```
=====
=== Feature Discretization ===
=====
```

Sample of discretized features:

	player_id	age_at_valuation	age_group \
1	10	27.310062	Prime (24-28)
2	10	28.098563	Experienced (29-33)
3	10	29.032170	Experienced (29-33)
4	10	29.987680	Experienced (29-33)
5	10	31.225188	Experienced (29-33)
6	10	31.912389	Experienced (29-33)
7	10	33.054073	Veteran (34+)
8	10	34.067077	Veteran (34+)
9	10	35.028063	Veteran (34+)
10	10	36.076660	Veteran (34+)

	market_value_in_eur_valuation	value_tier
1	15000000	High (5-20M)
2	30000000	Elite (>20M)
3	23000000	Elite (>20M)
4	20000000	High (5-20M)
5	12000000	High (5-20M)
6	7500000	High (5-20M)
7	5000000	Medium (1-5M)
8	6000000	High (5-20M)
9	2000000	Medium (1-5M)
10	1000000	Low (<1M)

Age Group Distribution:

age_group	
Young (≤23)	57790
Prime (24-28)	75762
Experienced (29-33)	45140
Veteran (34+)	12461

Name: count, dtype: int64

Current Value Tier Distribution:

value_tier	
Low (<1M)	129374
Medium (1-5M)	43295
High (5-20M)	14788
Elite (>20M)	3696

Name: count, dtype: int64

Target Value Tier Distribution:

target_value_tier	
Low (<1M)	128623
Medium (1-5M)	43212
High (5-20M)	15238
Elite (>20M)	4080

Name: count, dtype: int64

We one-hot encode categorical columns used by the model.

```
In [14]: # One-hot encoding

categorical_cols = [
    "position",
    "sub_position",
    "foot",
    "domestic_competition_id_valuation"
]

merged_encoded = pd.get_dummies(merged_df, columns=categorical_cols, drop_first=True)

print("encoded shape:", merged_encoded.shape)
```

encoded shape: (191153, 70)

We build the feature matrix `X` and target `y`. We explicitly remove columns that would leak information (current valuation) and identifiers.

```
In [15]: # Build X/y (no leakage)

y = merged_encoded["market_value_next_year"]

drop_cols = [
    "market_value_next_year",
    "market_value_in_eur_valuation",
    "player_id",
    "date",
    "date_of_birth",
    "year"
]
X = merged_encoded.drop(columns=[c for c in drop_cols if c in merged_encoded.columns])

# Ensure numeric + dummy (bool) columns are kept
X = X.select_dtypes(include=[np.number, "bool"]).copy()

# Convert bool dummies to 0/1 ints for sklearn
bool_cols = X.select_dtypes(include=["bool"]).columns
if len(bool_cols) > 0:
    X[bool_cols] = X[bool_cols].astype(int)

# Clean inf/NaN
X = X.replace([np.inf, -np.inf], np.nan)
X = X.fillna(X.median(numeric_only=True))

print("X shape:", X.shape)
print("y shape:", y.shape)
print("Any NaNs in X?", X.isna().any().any())
```

X shape: (191153, 61)
y shape: (191153,)
Any NaNs in X? False

2. Data Analysis and Visualization

We explore distributions, centrality measures, and relationships between features and the target.

```
In [16]: # Summary Statistics
print("=" * 60)
print("=== Measures of Centrality and Distribution ===")
print("=" * 60)

print("\n MODEL VARIABLES:")
print("Dependent Variable (Target): market_value_next_year")
print("Independent Variables: age_at_valuation, height_in_cm, position feature
print("                                market_value_lag_1, market_value_lag_2, market_v
print("                                market_value_mean_3, performance statistics (goa
print("                                minutes_played, goals_per_90), and other histori

# Key numerical variables for analysis
key_vars = ['market_value_in_eur_valuation', 'market_value_next_year',
            'age_at_valuation', 'height_in_cm',
            'market_value_lag_1', 'market_value_trend',
            'total_goals', 'total_assists', 'total_minutes',
            'goals_per_90', 'assists_per_90', 'num_appearances']

print("=" * 60)
print("Summary Statistics for Key Variables")
print("=" * 60)
summary_stats = merged_df[key_vars].describe()
display(summary_stats)

print("\n" + "=" * 60)
print("Additional Measures of Centrality")
print("=" * 60)
for var in key_vars:
    print(f"\n{var}:")
    print(f"  Mean:      €{merged_df[var].mean():.2f}" if 'value' in var or 'm
        else f"  Mean:      {merged_df[var].mean():.2f}")
    print(f"  Median:    €{merged_df[var].median():.2f}" if 'value' in var or
        else f"  Median:    {merged_df[var].median():.2f}")
    mode_val = merged_df[var].mode()
    print(f"  Mode:      {mode_val[0]:.2f}" if len(mode_val) > 0 else "  Mode:
    print(f"  Std Dev:  €{merged_df[var].std():.2f}" if 'value' in var or 'ma
        else f"  Std Dev:  {merged_df[var].std():.2f}")
    print(f"  Skewness: {merged_df[var].skew():.2f}")
    print(f"  Kurtosis: {merged_df[var].kurtosis():.2f}") # Peakness of probab

print("\n" + "=" * 60)
```


=====

=== Measures of Centrality and Distribution ===

=====

MODEL VARIABLES:

Dependent Variable (Target): market_value_next_year

Independent Variables: age_at_valuation, height_in_cm, position features,
market_value_lag_1, market_value_lag_2, market_value_trend,
market_value_mean_3, performance statistics (goals, assists,
minutes_played, goals_per_90), and other historical features

=====

Summary Statistics for Key Variables

=====

	market_value_in_eur_valuation	market_value_next_year	age_at_valuation
count	1.911530e+05	1.911530e+05	191153.000000
mean	2.402434e+06	2.514703e+06	25.826930
std	6.666086e+06	7.029003e+06	4.333907
min	1.000000e+04	1.000000e+04	15.619439
25%	2.500000e+05	2.500000e+05	22.414784
50%	6.000000e+05	6.000000e+05	25.355236
75%	1.750000e+06	1.800000e+06	28.821355
max	2.000000e+08	2.000000e+08	44.898015

=====

Additional Measures of Centrality

=====

market_value_in_eur_valuation:

Mean: €2,402,434.33
Median: €600,000.00
Mode: 300000.00
Std Dev: €6,666,085.54
Skewness: 8.40
Kurtosis: 112.91

market_value_next_year:

Mean: €2,514,703.43
Median: €600,000.00
Mode: 300000.00
Std Dev: €7,029,002.59
Skewness: 8.30
Kurtosis: 110.40

age_at_valuation:

Mean: 25.83
Median: 25.36
Mode: 24.38
Std Dev: 4.33
Skewness: 0.45
Kurtosis: -0.39

height_in_cm:

Mean: 182.39
Median: 183.00
Mode: 183.00
Std Dev: 6.63
Skewness: -0.62
Kurtosis: 13.39

market_value_lag_1:

Mean: €2,211,963.49
Median: €500,000.00
Mode: 300000.00
Std Dev: €6,273,373.52
Skewness: 8.74
Kurtosis: 123.26

market_value_trend:

Mean: €190,783.37
Median: €0.00
Mode: 0.00
Std Dev: €2,587,263.81
Skewness: 5.69
Kurtosis: 165.10

total_goals:

Mean: 2.71

Median: 0.00
Mode: 0.00
Std Dev: 11.22
Skewness: 10.67
Kurtosis: 204.45

total_assists:
Mean: 2.16
Median: 0.00
Mode: 0.00
Std Dev: 7.60
Skewness: 7.17
Kurtosis: 80.92

total_minutes:
Mean: 1769.78
Median: 0.00
Mode: 0.00
Std Dev: 4312.93
Skewness: 3.41
Kurtosis: 13.82

goals_per_90:
Mean: 0.04
Median: 0.00
Mode: 0.00
Std Dev: 0.17
Skewness: 110.87
Kurtosis: 27423.35

assists_per_90:
Mean: 0.03
Median: 0.00
Mode: 0.00
Std Dev: 0.26
Skewness: 260.07
Kurtosis: 84999.61

num_appearances:
Mean: 24.50
Median: 0.00
Mode: 0.00
Std Dev: 57.44
Skewness: 3.19
Kurtosis: 11.80

=====

```
In [17]: merged_df[["starts", "lineup_matches", "start_ratio", "captain_matches"]].desc
```

Out[17]:

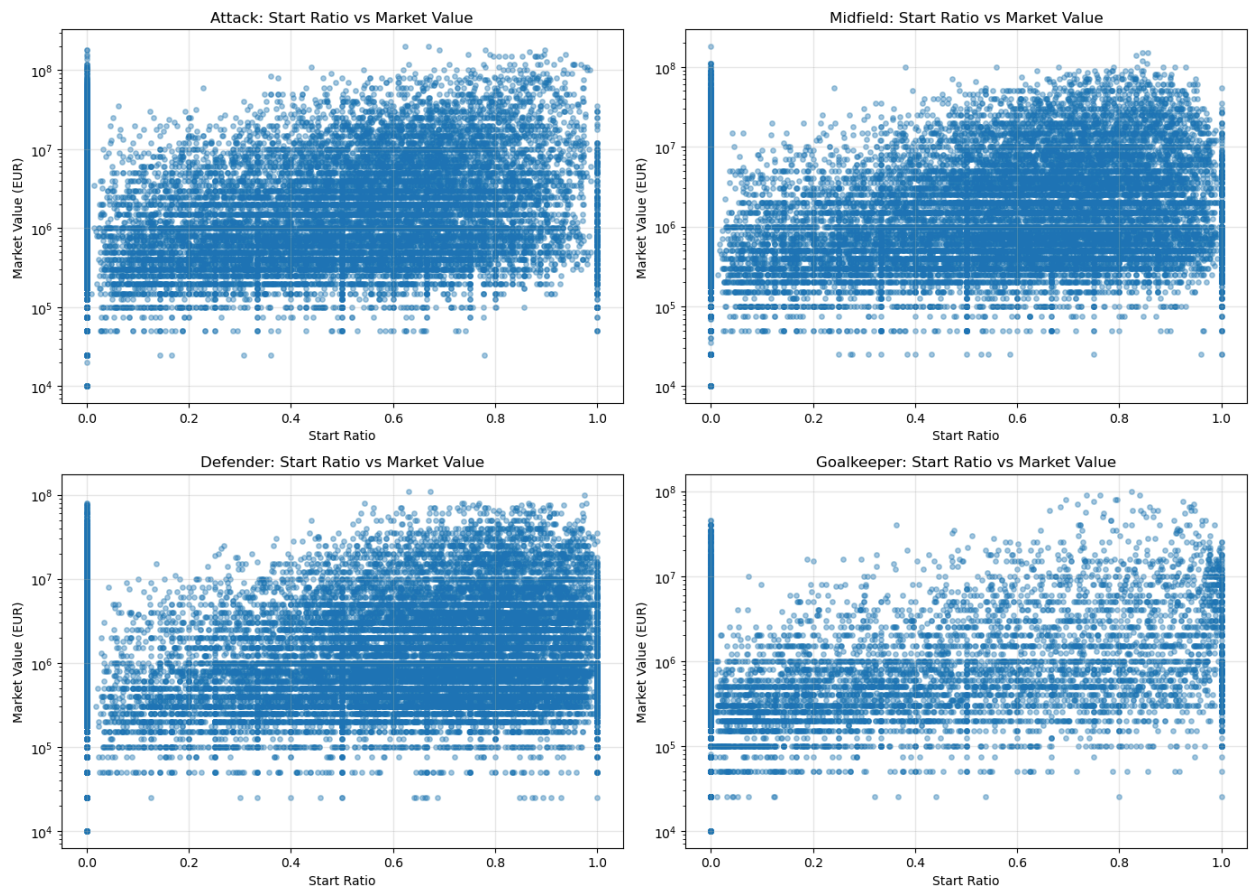
	starts	lineup_matches	start_ratio	captain_matches
count	191153.000000	191153.000000	191153.000000	191153.000000
mean	16.512720	26.249962	0.179066	1.201378
std	40.738198	57.938971	0.304089	9.106114
min	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000	0.000000
75%	7.000000	20.000000	0.327869	0.000000
max	418.000000	490.000000	1.000000	359.000000

```
In [18]: fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

for idx, pos in enumerate(["Attack", "Midfield", "Defender", "Goalkeeper"]):
    pos_data = merged_df[merged_df["position"] == pos]

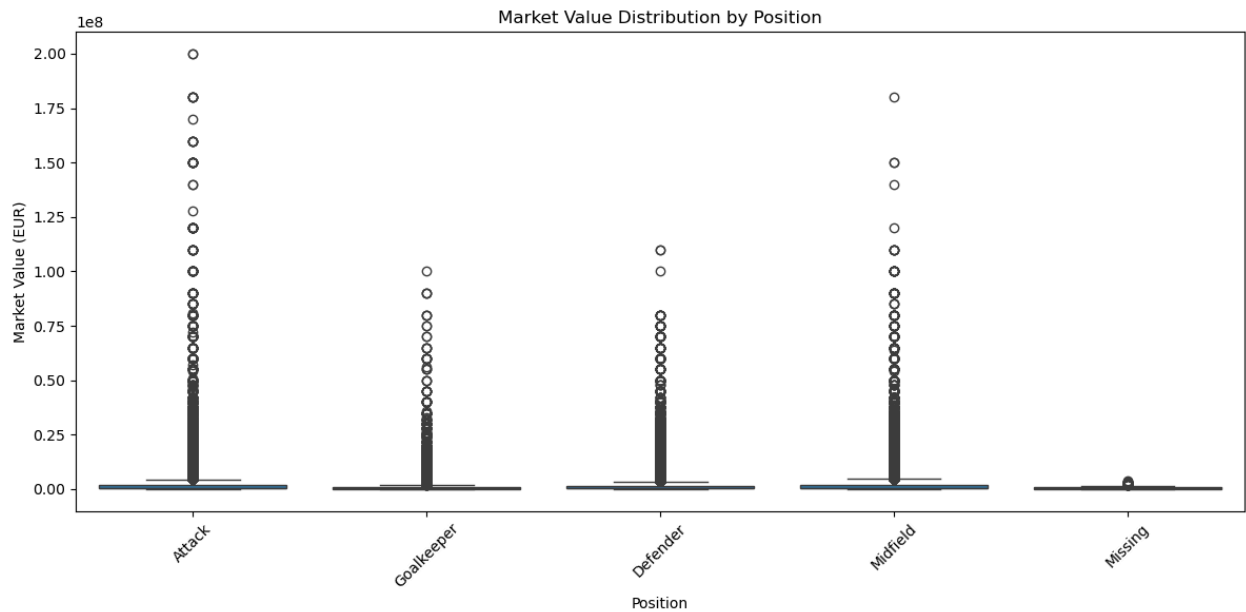
    axes[idx].scatter(
        pos_data["start_ratio"],
        pos_data["market_value_in_eur_valuation"],
        alpha=0.4, s=15
    )
    axes[idx].set_title(f"{pos}: Start Ratio vs Market Value")
    axes[idx].set_xlabel("Start Ratio")
    axes[idx].set_ylabel("Market Value (EUR)")
    axes[idx].set_yscale("log")
    axes[idx].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



```
In [19]: # Boxplot: Market Value by Position, colored by goal-scoring performance
import seaborn as sns

plt.figure(figsize=(12, 6))
sns.boxplot(data=merged_df, x='position', y='market_value_in_eur_valuation')
plt.xlabel('Position')
plt.ylabel('Market Value (EUR)')
plt.title('Market Value Distribution by Position')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
In [20]: #Faceted Analysis: Goals by Position (active players)
print("Creating position-specific analysis for Goals...")
print("Note: Filtering to players with >0 goals to show meaningful relationships")

# Get unique positions
positions = merged_df['position'].value_counts().index[:4]

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

for idx, pos in enumerate(positions):
    # Filter players with goals > 0
    pos_data = merged_df[(merged_df['position'] == pos) & (merged_df['total_goals'] > 0)]

    axes[idx].scatter(pos_data['total_goals'],
                      pos_data['market_value_in_eur_valuation'],
                      alpha=0.4, s=15, color='steelblue')

    axes[idx].set_xlabel('Total Goals (Season)', fontsize=10)
    axes[idx].set_ylabel('Market Value (EUR)', fontsize=10)
    axes[idx].set_title(f'{pos} - Goals vs Market Value\n(n={len(pos_data)} with >0 goals)',
                        fontsize=11, fontweight='bold')
    axes[idx].grid(True, alpha=0.3)

# Add trend line and calculate statistics
if len(pos_data) > 10:
    try:
        # Calculate correlation
        correlation = pos_data[['total_goals', 'market_value_in_eur_valuation']].corr().iloc[0,1]

        # Calculate slope (linear regression)
        z = np.polyfit(pos_data['total_goals'], pos_data['market_value_in_eur_valuation'], 1)
        slope = z[0]
        p = np.poly1d(z)
```

```

        # Plot trend line
        x_trend = np.linspace(pos_data['total_goals'].min(), pos_data['total_goals'].max(), 10)
        axes[idx].plot(x_trend, p(x_trend), "r--", alpha=0.6, linewidth=2)

        # Add text box with statistics
        textstr = f'Slope: €{slope:,.0f}/goal\nCorr: {correlation:.3f}'
        props = dict(boxstyle='round', facecolor='wheat', alpha=0.8)
        axes[idx].text(0.95, 0.95, textstr, transform=axes[idx].transAxes,
                      fontsize=9, verticalalignment='top', horizontalalignment='right',
                      bbox=props)
    except Exception as e:
        print(f"Could not calculate stats for {pos}: {e}")

plt.tight_layout()
plt.show()

# Print summary
print("\nData coverage after leakage fix:")
for pos in positions:
    total = len(merged_df[merged_df['position'] == pos])
    with_goals = len(merged_df[(merged_df['position'] == pos) & (merged_df['total_goals'] > 0)])
    print(f"    {pos}: {with_goals}/{total} ({with_goals/total*100:.1f}%) have goals")

```

Creating position-specific analysis for Goals...

Note: Filtering to players with >0 goals to show meaningful relationships

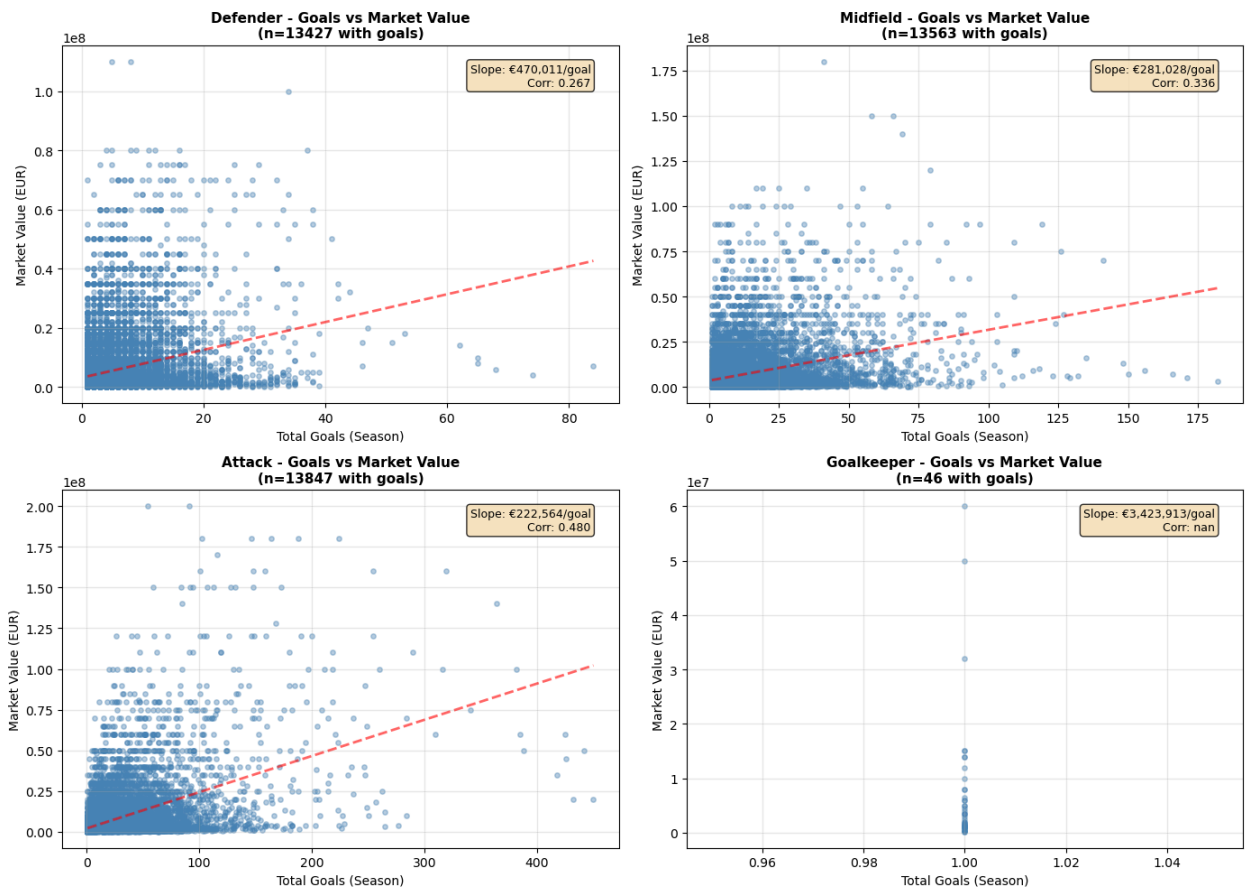
/var/folders/_3/vjdmg_yj15j8km8wyn2pr8c0000gn/T/ipykernel_20499/2705644840.py

y:31: RankWarning: Polyfit may be poorly conditioned

```

    z = np.polyfit(pos_data['total_goals'], pos_data['market_value_in_eur_valuation'], 1)

```



Data coverage after leakage fix:

Defender: 13427/62278 (21.6%) have goals before valuation

Midfield: 13563/55371 (24.5%) have goals before valuation

Attack: 13847/52178 (26.5%) have goals before valuation

Goalkeeper: 46/21015 (0.2%) have goals before valuation

```
In [21]: # Faceted Analysis: Assists by Position with Correlation Stats
print("Creating position-specific analysis for Assists...")

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

for idx, pos in enumerate(positions):
    pos_data = merged_df[merged_df['position'] == pos]

    axes[idx].scatter(pos_data['total_assists'],
                      pos_data['market_value_in_eur_valuation'],
                      alpha=0.4, s=15, color='seagreen')
    axes[idx].set_xlabel('Total Assists (Season)', fontsize=10)
    axes[idx].set_ylabel('Market Value (EUR)', fontsize=10)
    axes[idx].set_title(f'{pos} - Assists vs Market Value (n={len(pos_data)})',
                        fontsize=11, fontweight='bold')
    axes[idx].grid(True, alpha=0.3)

# Add trend line and calculate statistics
if len(pos_data) > 10:
    # Calculate correlation
```



```

correlation = pos_data[['total_assists', 'market_value_in_eur_valuation'])

# Calculate slope (linear regression)
z = np.polyfit(pos_data['total_assists'], pos_data['market_value_in_eur_valuation'], 1)
slope = z[0]
p = np.poly1d(z)

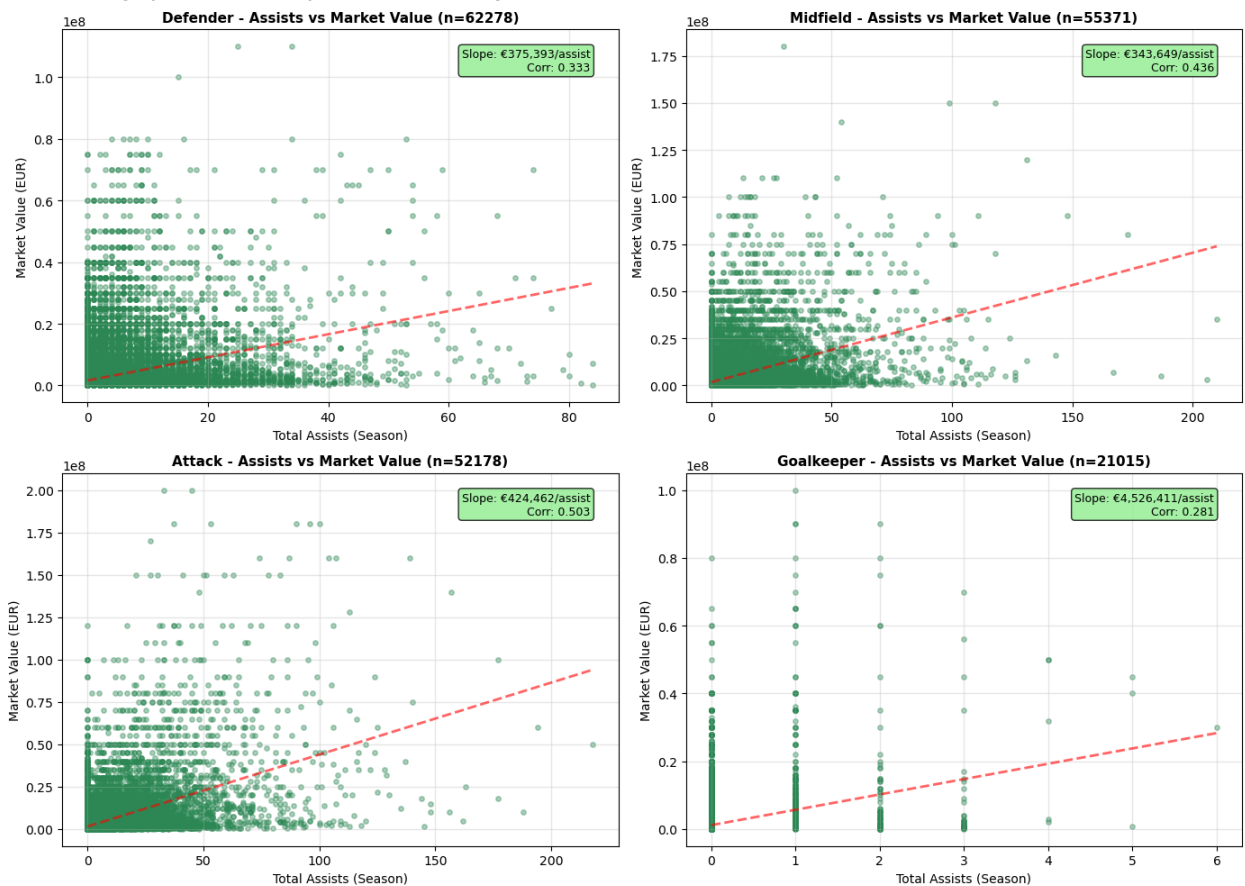
# Plot trend line
x_trend = np.linspace(pos_data['total_assists'].min(), pos_data['total_assists'].max(), 100)
axes[idx].plot(x_trend, p(x_trend), "r--", alpha=0.6, linewidth=2)

# Add text box with statistics in top right
textstr = f'Slope: €{slope:,.0f}/assist\nCorr: {correlation:.3f}'
props = dict(boxstyle='round', facecolor='lightgreen', alpha=0.8)
axes[idx].text(0.95, 0.95, textstr, transform=axes[idx].transAxes,
               fontsize=9, verticalalignment='top', horizontalalignment='right',
               bbox=props)

plt.tight_layout()
plt.show()

```

Creating position-specific analysis for Assists...



```

In [22]: # Goal contributions = goals + assists (season totals)
merged_df["total_goal_contributions"] = merged_df["total_goals"] + merged_df["total_assists"]

# Look at per-90 since minutes are very uneven (already made goal_contribution)

```

```

positions = ["Attack", "Midfield", "Defender", "Goalkeeper"]

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

for idx, pos in enumerate(positions):
    pos_data = merged_df[merged_df["position"] == pos].copy()

    # helpful: drop missing/inf just for plotting
    pos_data = pos_data.replace([np.inf, -np.inf], np.nan).dropna(
        subset=["total_goal_contributions", "market_value_in_eur_valuation"]
    )

    axes[idx].scatter(
        pos_data["total_goal_contributions"],
        pos_data["market_value_in_eur_valuation"],
        alpha=0.35, s=15
    )
    axes[idx].set_xlabel("Total Goal Contributions (Goals + Assists)")
    axes[idx].set_ylabel("Market Value (EUR)")
    axes[idx].set_title(f"{pos} – Goal Contributions vs Market Value (n={len(pos_data)})")
    axes[idx].grid(True, alpha=0.3)

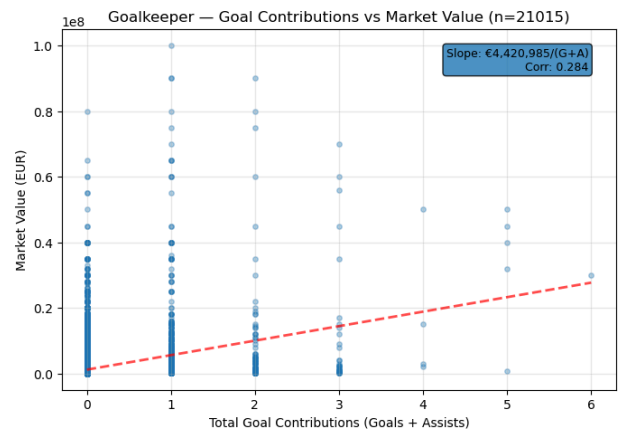
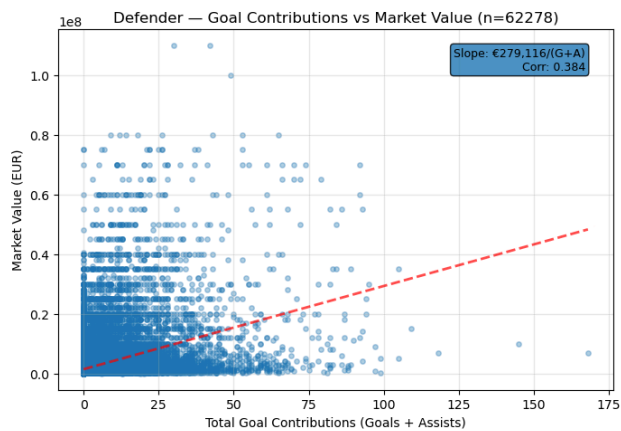
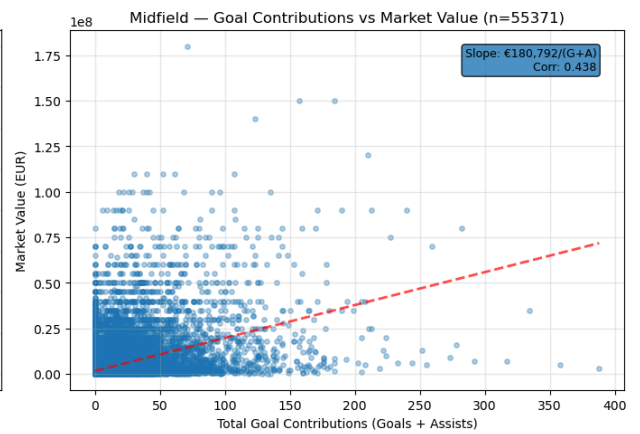
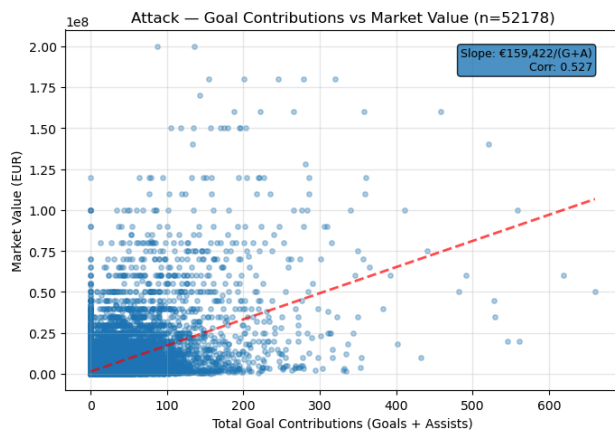
    if len(pos_data) > 10:
        corr = pos_data[["total_goal_contributions", "market_value_in_eur_valuation"]].corr()[0,1]
        z = np.polyfit(pos_data["total_goal_contributions"], pos_data["market_value_in_eur_valuation"], 1)
        slope = z[0]
        p = np.poly1d(z)

        x_trend = np.linspace(pos_data["total_goal_contributions"].min(),
                               pos_data["total_goal_contributions"].max(), 100)
        axes[idx].plot(x_trend, p(x_trend), "r--", alpha=0.7, linewidth=2)

        axes[idx].text(
            0.95, 0.95,
            f"Slope: €{slope:,.0f}/(G+A)\nCorr: {corr:.3f}",
            transform=axes[idx].transAxes,
            ha="right", va="top", fontsize=9,
            bbox=dict(boxstyle="round", alpha=0.8)
        )

plt.tight_layout()
plt.show()

```

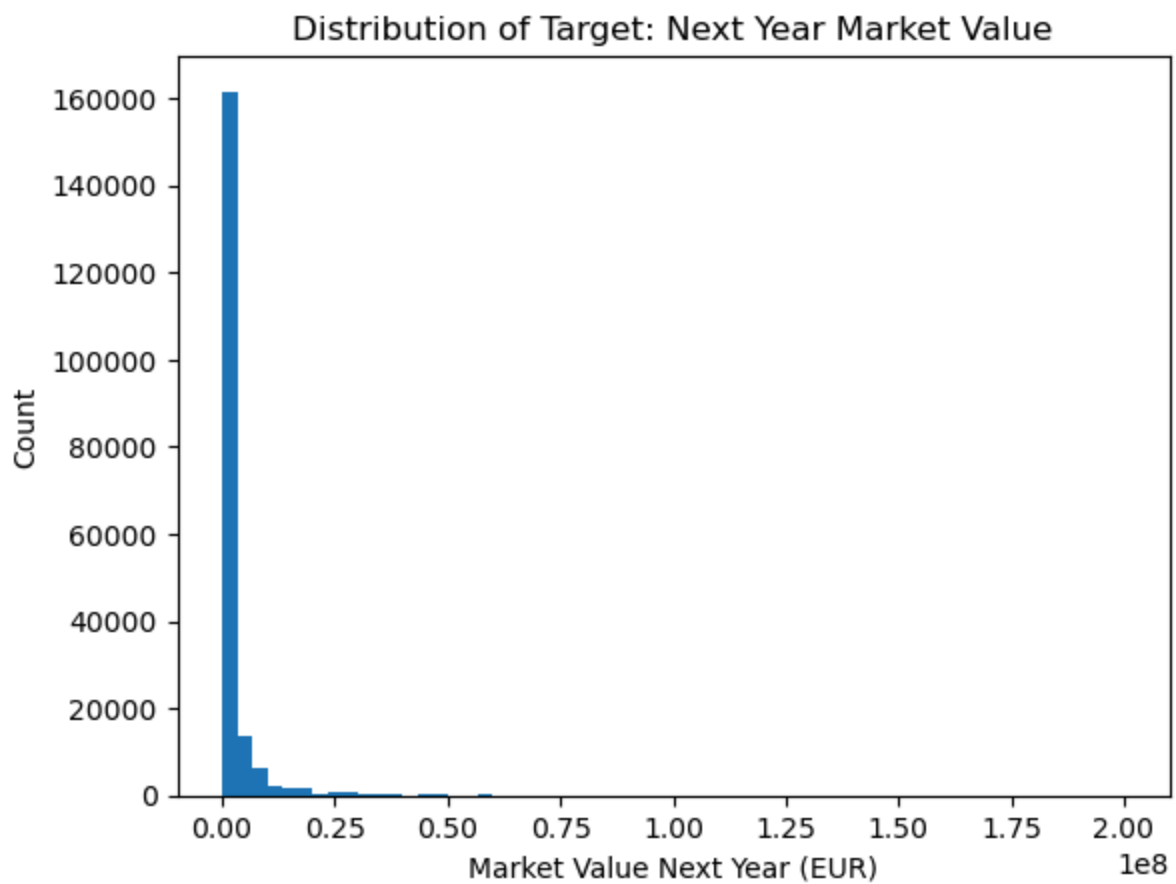


In [23]: *# Statistics*

```
display(y.describe())

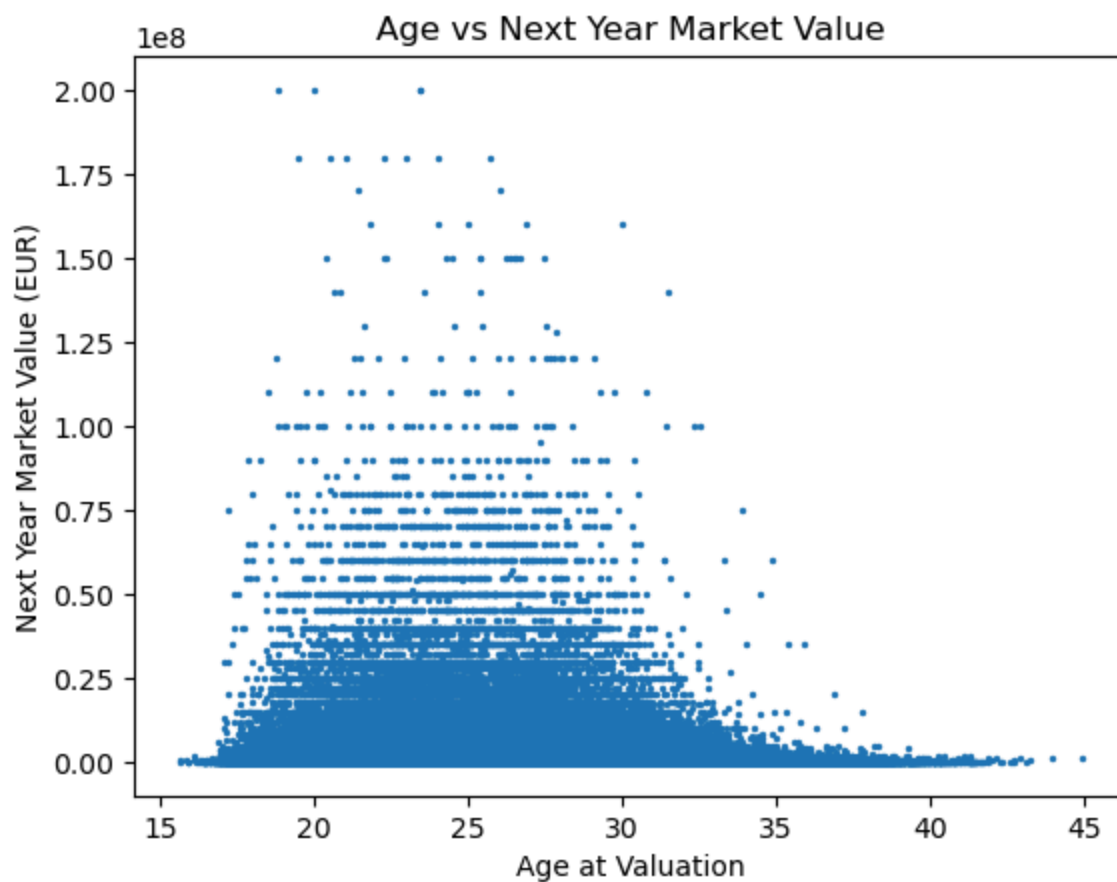
plt.figure()
plt.hist(y, bins=60)
plt.xlabel("Market Value Next Year (EUR)")
plt.ylabel("Count")
plt.title("Distribution of Target: Next Year Market Value")
plt.show()
```

```
count    1.911530e+05
mean     2.514703e+06
std      7.029003e+06
min      1.000000e+04
25%      2.500000e+05
50%      6.000000e+05
75%      1.800000e+06
max      2.000000e+08
Name: market_value_next_year, dtype: float64
```

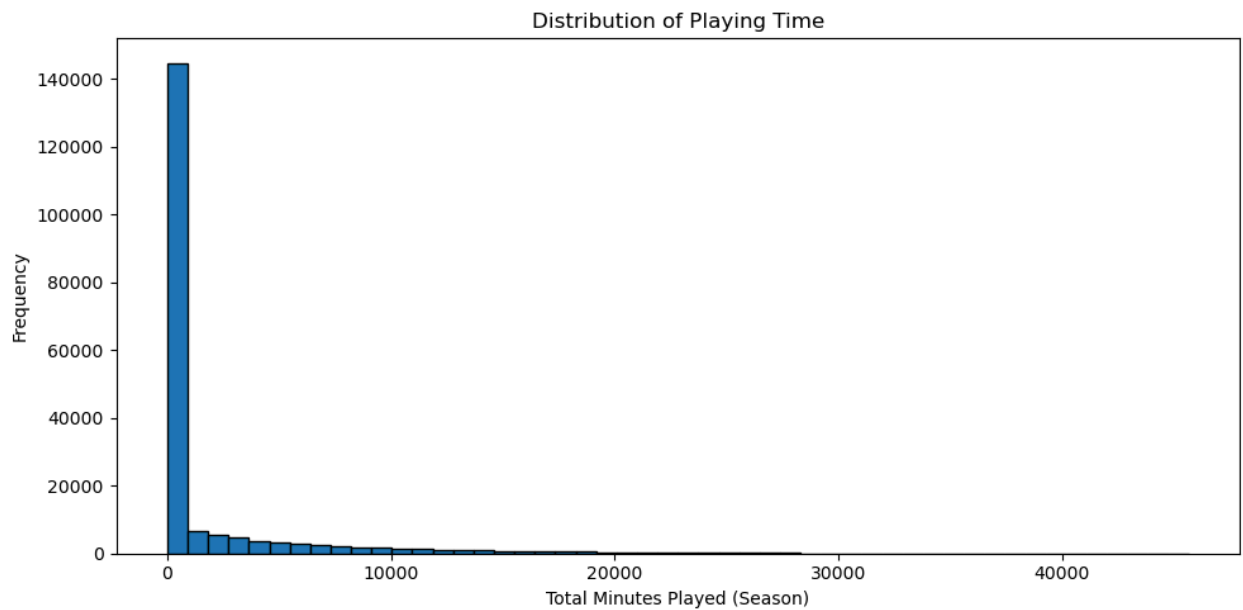


In [24]: *# age vs next-year value*

```
plt.figure()
plt.scatter(merged_df["age_at_valuation"], merged_df["market_value_next_year"])
plt.xlabel("Age at Valuation")
plt.ylabel("Next Year Market Value (EUR)")
plt.title("Age vs Next Year Market Value")
plt.show()
```



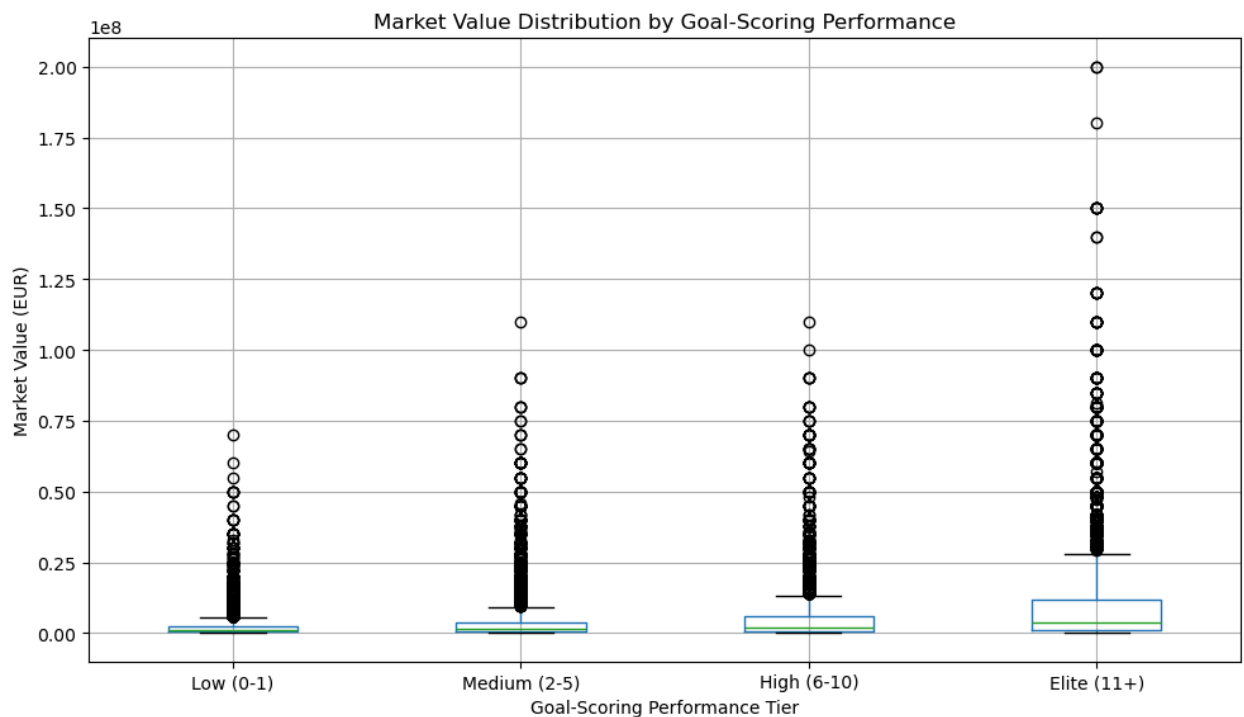
```
In [25]: # Minutes Played Distribution
plt.figure(figsize=(10, 5))
plt.hist(merged_df['total_minutes'], bins=50, edgecolor='black')
plt.xlabel('Total Minutes Played (Season)')
plt.ylabel('Frequency')
plt.title('Distribution of Playing Time')
plt.tight_layout()
plt.show()
```



```
In [26]: # Market Value by Performance Tier (boxplot)
# Create performance tiers
merged_df['performance_tier'] = pd.cut(
    merged_df['total_goals'],
    bins=[0, 1, 5, 10, 100],
    labels=['Low (0-1)', 'Medium (2-5)', 'High (6-10)', 'Elite (11+)']
)

plt.figure(figsize=(10, 6))
merged_df.boxplot(column='market_value_in_eur_valuation', by='performance_tier',
                  figsize=(10, 6))
plt.xlabel('Goal-Scoring Performance Tier')
plt.ylabel('Market Value (EUR)')
plt.title('Market Value Distribution by Goal-Scoring Performance')
plt.suptitle('') # Remove default title
plt.tight_layout()
plt.show()
```

<Figure size 1000x600 with 0 Axes>

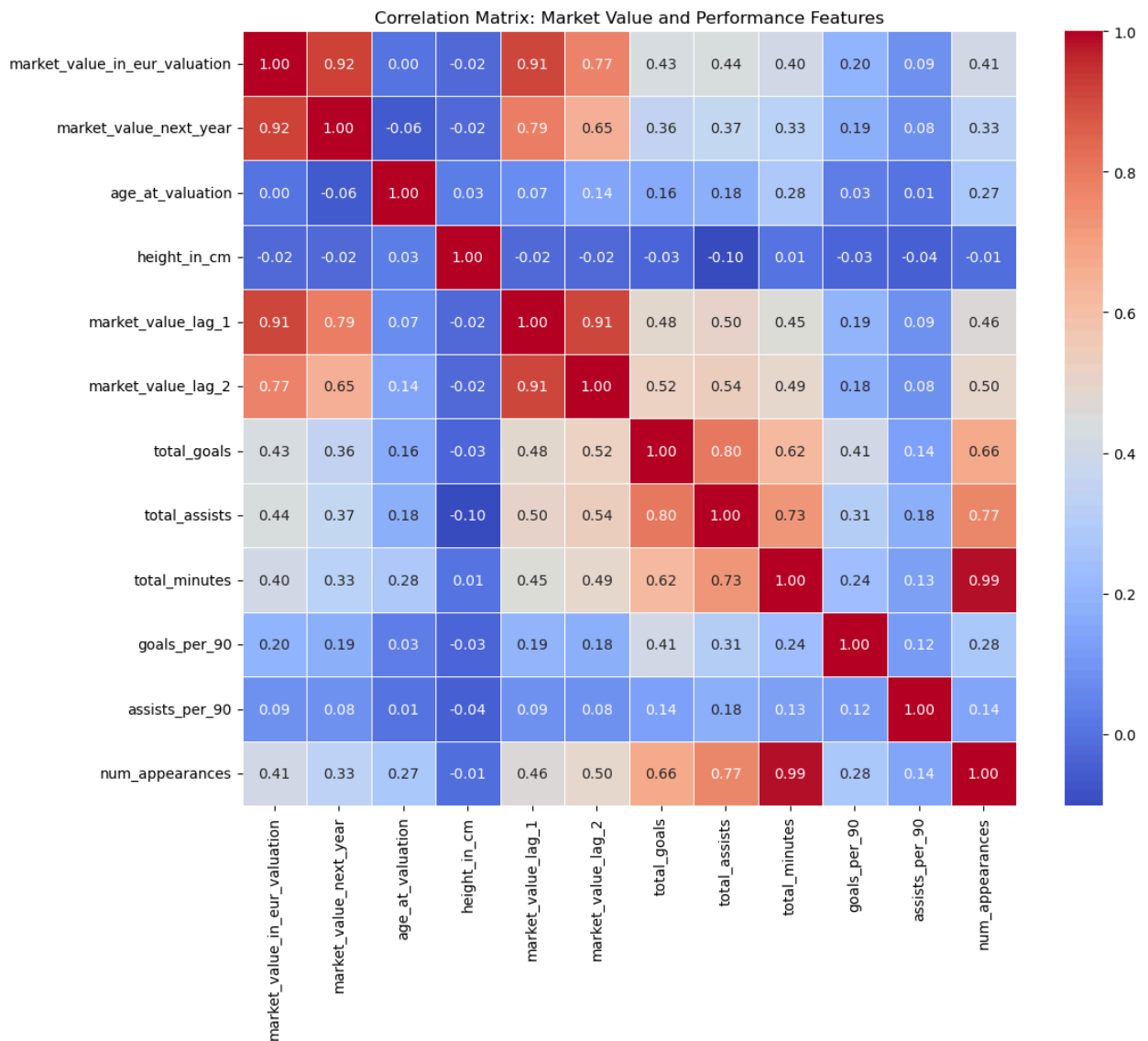


```
In [27]: # Correlation heatmap
import seaborn as sns

# Select key features including performance stats
corr_features = ['market_value_in_eur_valuation', 'market_value_next_year',
                 'age_at_valuation', 'height_in_cm',
                 'market_value_lag_1', 'market_value_lag_2',
                 'total_goals', 'total_assists', 'total_minutes',
                 'goals_per_90', 'assists_per_90', 'num_appearances']

corr_matrix = merged_df[corr_features].corr()

plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm',
            square=True, linewidths=0.5)
plt.title('Correlation Matrix: Market Value and Performance Features')
plt.tight_layout()
plt.show()
```



4. Data Analytics (Modeling)

This is a **supervised learning regression** problem. We compare multiple algorithms and evaluate them using MAE, RMSE, and R^2 . We also do time-aware train/test splitting to reduce leakage risk.

```
In [28]: # Train/Test split (time-aware)
# We preserve order (shuffle=False) because data is time-dependent.

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)

print("Train:", X_train.shape, y_train.shape)
print("Test:", X_test.shape, y_test.shape)
```

```
Train: (152922, 61) (152922,)
Test: (38231, 61) (38231,)
```


We compare:

- Linear Regression (baseline)
- ElasticNet (regularized linear)
- Random Forest (non-linear)
- KNN (distance-based)

For KNN and ElasticNet we benefit from scaling, so we use pipelines.

In [29]: *# Models + evaluation*

```
def evaluate_model(name, model, X_tr, y_tr, X_te, y_te):
    """
    Train model, evaluate on test set, and perform cross-validation
    """
    # Train the model
    model.fit(X_tr, y_tr)

    # Test set predictions
    pred = model.predict(X_te)

    # Cross-validation on training set (5-fold)
    print(f"\n{'='*60}")
    print(f"Cross-Validation: {name}")
    print(f"{'='*60}")

    cv_r2 = cross_val_score(model, X_tr, y_tr, cv=5, scoring='r2', n_jobs=-1)
    cv_mae = -cross_val_score(model, X_tr, y_tr, cv=5, scoring='neg_mean_absol
    cv_rmse = np.sqrt(-cross_val_score(model, X_tr, y_tr, cv=5, scoring='neg_m

    print(f"  R² scores:    {cv_r2}")
    print(f"  Mean R²:      {cv_r2.mean():.4f} (+/- {cv_r2.std():.4f})")
    print(f"  MAE scores:   {cv_mae}")
    print(f"  Mean MAE:     €{cv_mae.mean():.2f} (+/- €{cv_mae.std():.2f})")
    print(f"  RMSE scores:  {cv_rmse}")
    print(f"  Mean RMSE:    €{cv_rmse.mean():.2f} (+/- €{cv_rmse.std():.2f})")

    return {
        "Model": name,
        "MAE": mean_absolute_error(y_te, pred),
        "RMSE": np.sqrt(mean_squared_error(y_te, pred)),
        "R2": r2_score(y_te, pred),
        "CV_R2_Mean": cv_r2.mean(),
        "CV_R2_Std": cv_r2.std(),
        "CV_MAE_Mean": cv_mae.mean(),
        "CV_RMSE_Mean": cv_rmse.mean()
    }

models = []

# Baselines (no scaling needed)
models.append(("Linear Regression", LinearRegression()))
```

```

models.append(("Random Forest", RandomForestRegressor(
    n_estimators=150,
    max_depth=18,
    random_state=42,
    n_jobs=-1
)))

# Pipelines (imputer + scaler) helps for KNN/ElasticNet
elastic_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("model", ElasticNet(alpha=0.1, l1_ratio=0.5, random_state=42))
])

knn_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("model", KNeighborsRegressor(n_neighbors=10))
])

models.append(("ElasticNet (scaled)", elastic_pipe))
models.append(("KNN (scaled)", knn_pipe))

results = []
for name, model in models:
    results.append(evaluate_model(name, model, X_train, y_train, X_test, y_test))

results_df = pd.DataFrame(results).sort_values("MAE")
results_df

```

=====

Cross-Validation: Linear Regression

=====

R² scores: [0.72986698 0.68773923 0.69088713 0.64785225 0.64652585]
Mean R²: 0.6806 (+/- 0.0310)
MAE scores: [1577989.72199942 1324551.26725565 1316378.88945533 1454189.2484
976
1605435.4141858]
Mean MAE: €1,455,708.91 (+/- €121,645.48)
RMSE scores: [3162637.27898362 2975500.4382755 3382756.74327769 3993066.1271
4598
4056311.19214229]
Mean RMSE: €3,514,054.36 (+/- €436,867.97)

=====

Cross-Validation: Random Forest

=====

R² scores: [0.77094966 0.76675074 0.76710589 0.74003836 0.70807617]
Mean R²: 0.7506 (+/- 0.0239)
MAE scores: [1207242.02482667 1018677.68556404 1023876.3686598 1179044.1671
5047
1322435.20176775]
Mean MAE: €1,150,255.09 (+/- €115,763.99)
RMSE scores: [2912232.387914 2571647.49285639 2936239.52635532 3430825.8894
9673
3686270.59163278]
Mean RMSE: €3,107,443.18 (+/- €398,525.41)

=====

Cross-Validation: ElasticNet (scaled)

=====

R² scores: [0.72206634 0.68400424 0.68631945 0.64651966 0.64090865]
Mean R²: 0.6760 (+/- 0.0296)
MAE scores: [1596159.56210938 1336772.94305921 1333800.63223547 1486639.3687
7986
1648394.80378832]
Mean MAE: €1,480,353.46 (+/- €129,446.17)
RMSE scores: [3207976.13074211 2993242.69111167 3407658.16065616 4000614.1904
1627
4088414.37314839]
Mean RMSE: €3,539,581.11 (+/- €433,502.17)

=====

Cross-Validation: KNN (scaled)

=====

R² scores: [0.57933399 0.6032869 0.632734 0.60617194 0.59839276]
Mean R²: 0.6040 (+/- 0.0172)
MAE scores: [1793460.12754618 1415408.22298512 1353964.5402825 1466594.5592
4666
1594016.07049438]
Mean MAE: €1,524,688.70 (+/- €155,889.87)
RMSE scores: [3946655.91398886 3353818.64500577 3687250.03527396 4222769.4197
1887
4323676.56636962]

Mean RMSE: €3,906,834.12 (+/- €354,739.85)

Out[29]:	Model	MAE	RMSE	R2	CV_R2_Mean	CV_R2_Std	C
1	Random Forest	1.810234e+06	5.288976e+06	0.691578	0.750584	0.023947	1
3	KNN (scaled)	2.126536e+06	6.222347e+06	0.573116	0.603984	0.017151	1
0	Linear Regression	2.127443e+06	5.743761e+06	0.636257	0.680574	0.031040	1
2	ElasticNet (scaled)	2.185338e+06	5.890054e+06	0.617492	0.675964	0.029642	1

We use `TimeSeriesSplit` to evaluate models with time-aware cross-validation. This avoids mixing future information into training folds.

```
In [30]: # TimeSeries cross-validation (on a subset for speed)

tscv = TimeSeriesSplit(n_splits=5)

# For speed, sample a subset
sample_size = min(120000, len(X_train))
X_cv = X_train.iloc[:sample_size]
y_cv = y_train.iloc[:sample_size]

# Use MAE (negative in sklearn scoring conventions)
def cv_mae(model):
    scores = cross_val_score(
        model, X_cv, y_cv,
        cv=tscv,
        scoring="neg_mean_absolute_error",
        n_jobs=-1
    )
    return -scores.mean(), scores.std()

cv_rows = []
for name, model in models:
    mean_mae, std_mae = cv_mae(model)
    cv_rows.append({"Model": name, "CV_MAE_mean": mean_mae, "CV_MAE_std": std_mae})

cv_df = pd.DataFrame(cv_rows).sort_values("CV_MAE_mean")
cv_df
```

Out[30]:	Model	CV_MAE_mean	CV_MAE_std
1	Random Forest	1.164490e+06	50877.286810
0	Linear Regression	1.390831e+06	63387.240003
2	ElasticNet (scaled)	1.436050e+06	64503.661125
3	KNN (scaled)	1.592343e+06	169142.210753

We inspect which features contribute most to predictions (interpretability).

```
In [31]: # Feature importance (Random Forest)

# Train Random Forest
rf = RandomForestRegressor(
    n_estimators=150,
    max_depth=18,
    random_state=42,
    n_jobs=-1
)
rf.fit(X_train, y_train)

# Compute importances
importances = pd.Series(
    rf.feature_importances_,
    index=X_train.columns
).sort_values(ascending=False)

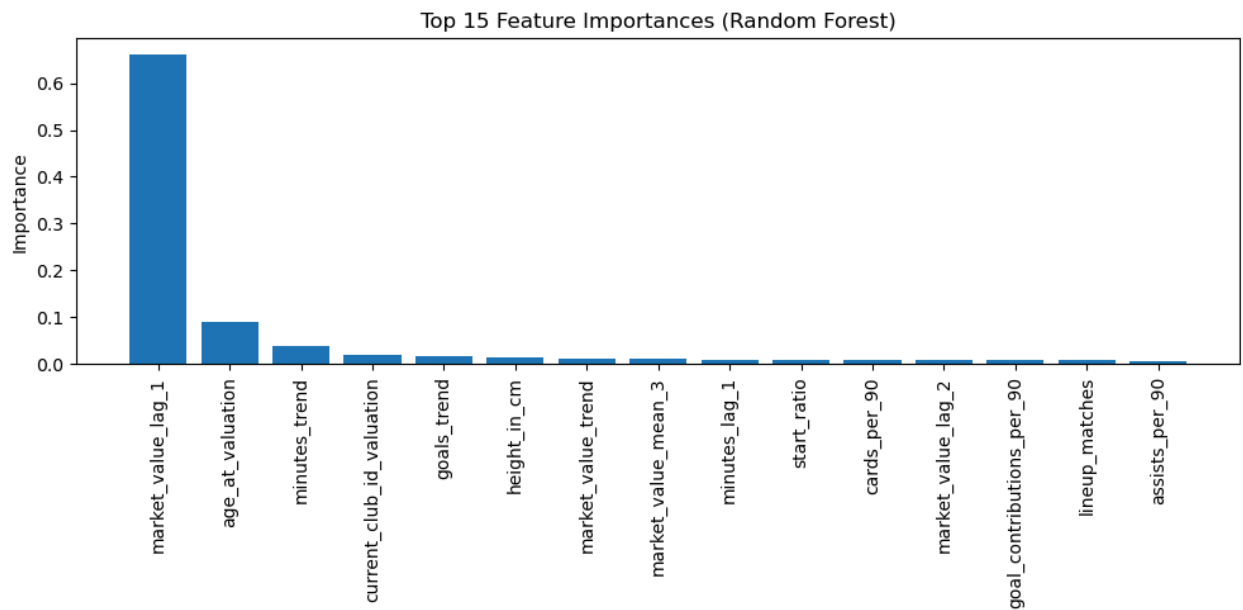
top_k = 15
display(importances.head(top_k))

# Plot feature importances
top_feats = importances.head(top_k)

x = np.arange(len(top_feats))
y = top_feats.to_numpy(dtype=float)

plt.figure(figsize=(10, 5))
plt.bar(x, y)
plt.xticks(x, top_feats.index, rotation=90)
plt.ylabel("Importance")
plt.title(f"Top {top_k} Feature Importances (Random Forest)")
plt.tight_layout()
plt.show()
```

market_value_lag_1	0.662409
age_at_valuation	0.089592
minutes_trend	0.038596
current_club_id_valuation	0.019426
goals_trend	0.017501
height_in_cm	0.012633
market_value_trend	0.011208
market_value_mean_3	0.010110
minutes_lag_1	0.009747
start_ratio	0.008737
cards_per_90	0.008236
market_value_lag_2	0.007871
goal_contributions_per_90	0.007281
lineup_matches	0.007224
assists_per_90	0.007019
dtype: float64	



```
In [32]: # Analyze top performing features
print("=" * 60)
print("Top 20 Feature Importances (Random Forest)")
print("=" * 60)

# Get feature importances from your trained RF model
rf = RandomForestRegressor(n_estimators=150, max_depth=18, random_state=42, n_
rf.fit(X_train, y_train)

importances = pd.Series(rf.feature_importances_, index=X_train.columns).sort_v

print("\nTop 20 Features:")
display(importances.head(20))

# Highlight performance features
print("\n Performance-Related Features in Top 20:")
perf_feature_keywords = ['goal', 'assist', 'minutes', 'appearance', 'cards']
top_20_features = importances.head(20)
for feat in top_20_features.index:
    if any(keyword in feat.lower() for keyword in perf_feature_keywords):
        print(f" - {feat}: {top_20_features[feat]:.4f}")
```

```
=====
Top 20 Feature Importances (Random Forest)
=====
```

Top 20 Features:

market_value_lag_1	0.662409
age_at_valuation	0.089592
minutes_trend	0.038596
current_club_id_valuation	0.019426
goals_trend	0.017501
height_in_cm	0.012633
market_value_trend	0.011208
market_value_mean_3	0.010110
minutes_lag_1	0.009747
start_ratio	0.008737
cards_per_90	0.008236
market_value_lag_2	0.007871
goal_contributions_per_90	0.007281
lineup_matches	0.007224
assists_per_90	0.007019
goals_per_90	0.006238
goals_per_90_lag_1	0.005662
total_yellow_cards	0.005267
total_assists	0.005189
num_appearances	0.005186

dtype: float64

Performance-Related Features in Top 20:

- minutes_trend: 0.0386
- goals_trend: 0.0175
- minutes_lag_1: 0.0097
- cards_per_90: 0.0082
- goal_contributions_per_90: 0.0073
- assists_per_90: 0.0070
- goals_per_90: 0.0062
- goals_per_90_lag_1: 0.0057
- total_yellow_cards: 0.0053
- total_assists: 0.0052
- num_appearances: 0.0052

A quick diagnostic plot to visually assess model fit.

In [33]: *# Predicted vs actual*

```
best_model_name = results_df.iloc[0]["Model"]
best_model = dict(models)[best_model_name]
best_model.fit(X_train, y_train)
pred = best_model.predict(X_test)

min_val = min(y_test.min(), pred.min())
max_val = max(y_test.max(), pred.max())

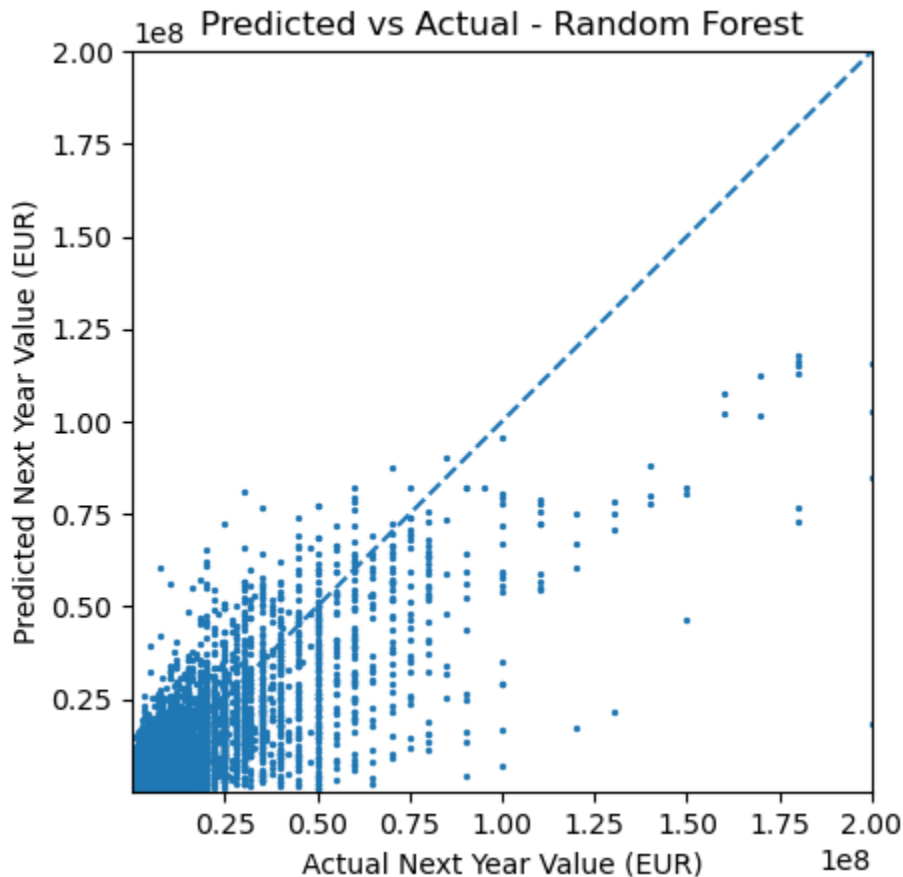
plt.figure()
plt.scatter(y_test, pred, s=2)

plt.xlim(min_val, max_val)
plt.ylim(min_val, max_val)

plt.gca().set_aspect("equal", adjustable="box")
```

```
plt.plot([min_val,max_val],[min_val,max_val],linestyle='--')

plt.xlabel("Actual Next Year Value (EUR)")
plt.ylabel("Predicted Next Year Value (EUR)")
plt.title(f"Predicted vs Actual - {best_model_name}")
plt.show()
```



In [34]: *# Predicted vs Actual*

```
final_model_name = results_df.iloc[0]["Model"]
final_model = dict(models)[final_model_name]

final_model.fit(X_train, y_train)
pred = final_model.predict(X_test)

# Convert to numpy arrays and remove non-positive values for log scaling
y_true = np.asarray(y_test).astype(float)
y_pred = np.asarray(pred).astype(float)

mask = (y_true > 0) & (y_pred > 0) & np.isfinite(y_true) & np.isfinite(y_pred)
y_true = y_true[mask]
y_pred = y_pred[mask]

# 1) HEXBIN DENSITY PLOT (final when there are many points)
plt.figure(figsize=(8, 6))
```



```

hb = plt.hexbin(y_true, y_pred, gridsize=60, bins='log') # density in log bins
plt.colorbar(hb, label="Log(count)")

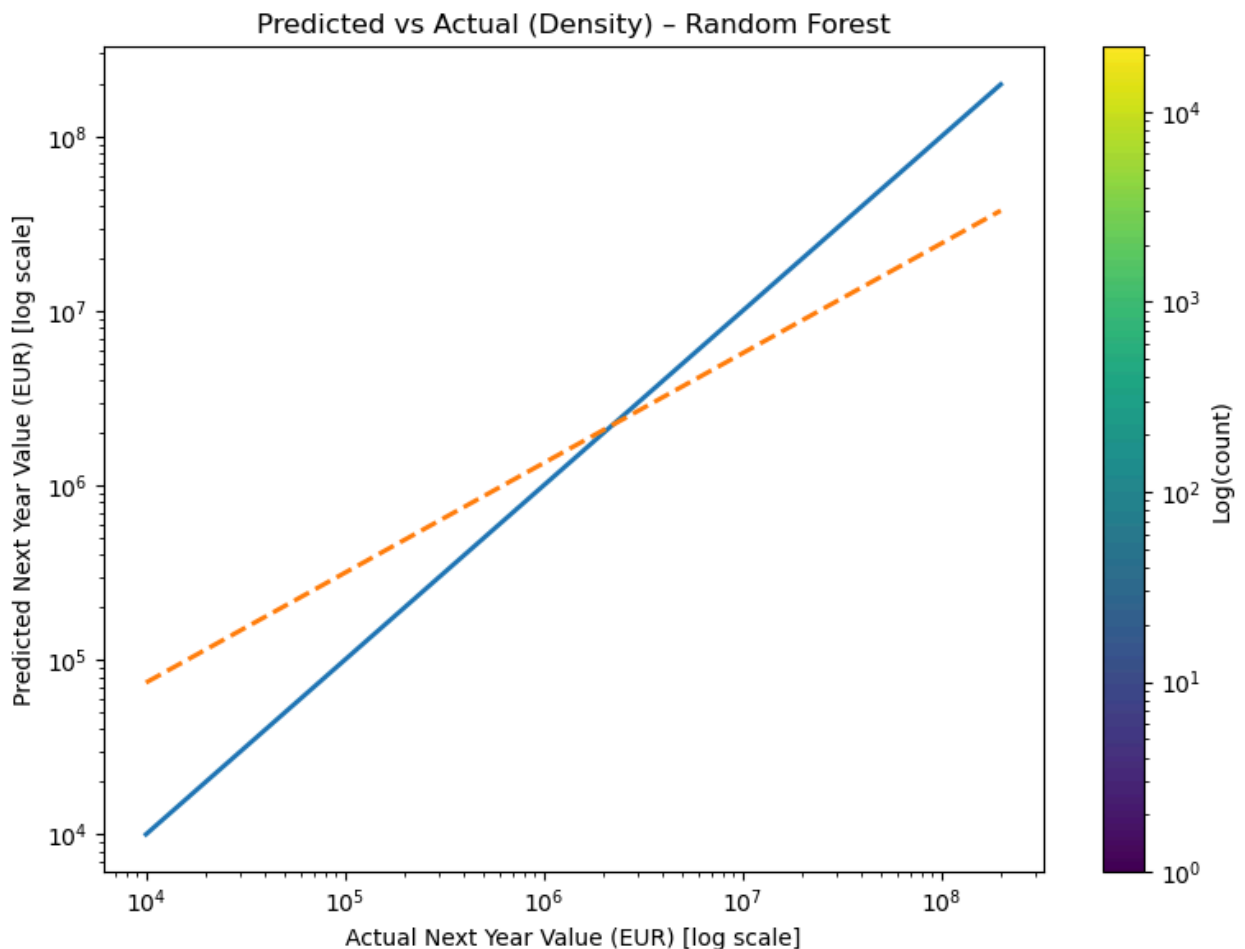
# Log scales (makes long-tail readable)
plt.xscale("log")
plt.yscale("log")

# 45-degree line (perfect predictions)
min_v = min(y_true.min(), y_pred.min())
max_v = max(y_true.max(), y_pred.max())
plt.plot([min_v, max_v], [min_v, max_v], linewidth=2)

# Fit line to log10 values to show overall calibration
logx = np.log10(y_true)
logy = np.log10(y_pred)
m, b = np.polyfit(logx, logy, 1)
x_line = np.logspace(np.log10(min_v), np.log10(max_v), 200)
y_line = 10**(m*np.log10(x_line) + b)
plt.plot(x_line, y_line, linestyle="--", linewidth=2)

plt.xlabel("Actual Next Year Value (EUR) [log scale]")
plt.ylabel("Predicted Next Year Value (EUR) [log scale]")
plt.title(f"Predicted vs Actual (Density) – {final_model_name}")
plt.tight_layout()
plt.show()

```



```

In [35]: import numpy as np
import matplotlib.pyplot as plt

# Data (means + std)
models = ["Linear Regression", "Random Forest", "Elastic Net", "KNN"]

rmse_mean = np.array([3514054.36, 3107443.18, 3539581.11, 3906834.12])
rmse_std = np.array([ 436867.97,  398525.41,  433502.17,  354739.85])

mae_mean = np.array([1455708.91, 1150255.09, 1480353.46, 1524688.70])
mae_std = np.array([ 121645.48,  115763.99,  129446.17,  155889.87])

r2_mean = np.array([0.6806, 0.7506, 0.6760, 0.6040])
r2_std = np.array([0.0310, 0.0239, 0.0296, 0.0172])

rmse_m = rmse_mean / 1e6
rmse_s = rmse_std / 1e6
mae_m = mae_mean / 1e6
mae_s = mae_std / 1e6

# Plot (grouped bars)
x = np.arange(len(models))
width = 0.24

fig, ax1 = plt.subplots(figsize=(12, 6))

# Colors by metric
c_rmse = "#e76f51"
c_mae = "#2a9d8f"
c_r2 = "#264653"

# Left axis: MAE + RMSE (EUR, millions)
b_mae = ax1.bar(x - width, mae_m, width, yerr=mae_s, capsize=4, label="MAE (€M)")
b_rmse = ax1.bar(x, rmse_m, width, yerr=rmse_s, capsize=4, label="RMSE (€M)")

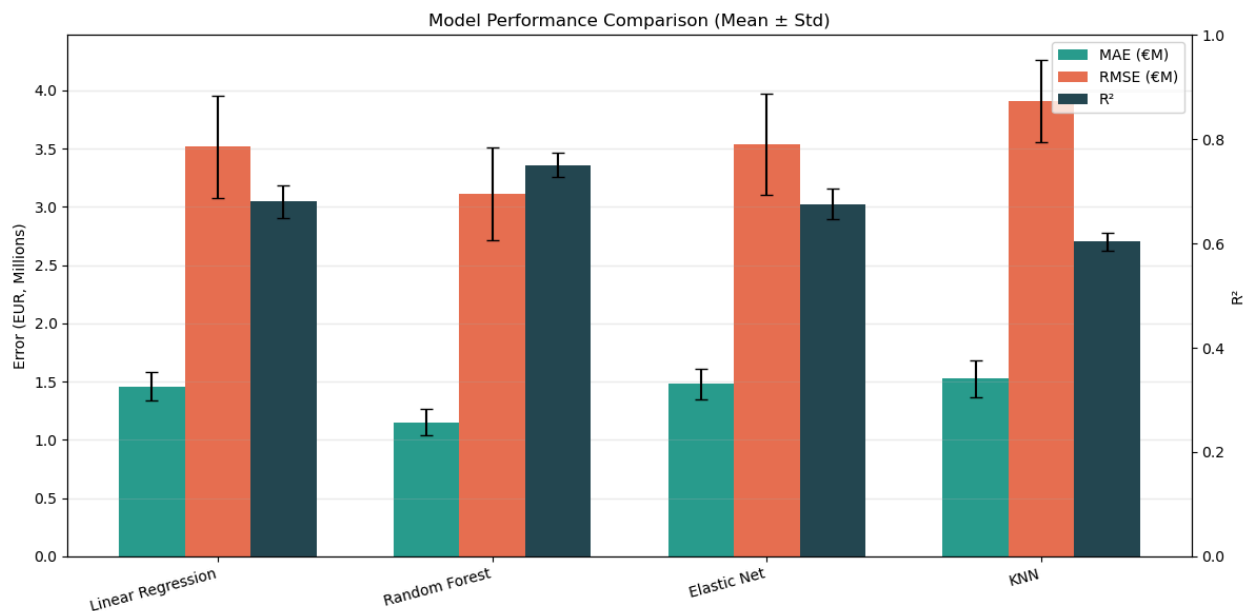
ax1.set_ylabel("Error (EUR, Millions)")
ax1.set_xticks(x)
ax1.set_xticklabels(models, rotation=15, ha="right")
ax1.grid(axis="y", alpha=0.25)

# Right axis: R²
ax2 = ax1.twinx()
b_r2 = ax2.bar(x + width, r2_mean, width, yerr=r2_std, capsize=4, label="R²",)
ax2.set_ylabel("R²")
ax2.set_ylim(0, 1.0)

# Title + combined legend
plt.title("Model Performance Comparison (Mean ± Std)")
handles1, labels1 = ax1.get_legend_handles_labels()
handles2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(handles1 + handles2, labels1 + labels2, loc="upper right")

```

```
plt.tight_layout()
plt.show()
```



5. Summary / Conclusions

- We framed the task as **supervised regression** to predict next-year market value.
- We used **time-aware feature engineering** (lags, trends, rolling means) and a **time-aware train/test split** to reduce leakage risk.
- We compared multiple models and evaluated them with **MAE, RMSE, and R²**, then validated via **TimeSeriesSplit** cross-validation.